Stratified Software Development

A Refinement–DAG Calculus for Agent–Authored Systems

Section titled “A Refinement–DAG Calculus for Agent–Authored Systems”

Formal Technical Report · November 2026

Abstract

Section titled “Abstract”

Autonomous coding agents fail in three recurring ways. They leap from a requirement to code and invent the missing design; they lose the rationale for load–bearing choices and later weaken tests to make builds pass; and they leave humans with no trustworthy intermediate view of what the system does. This report presents a refinement–DAG calculus that structures software development as an acyclic graph of verified behavior inclusions. A development record assigns each cell its active intents, support provenance, contextual models, state machines, a decomposition into child contracts, and evidence. One relation — behavior inclusion under a typed edge abstraction map — serves for intent satisfaction, child contract refinement, connector decomposition, and code realization. Stratification is the partial–order acyclicity of the Refinement DAG from root specifications to executable leaves: it turns support evaluation into an acyclic topological evaluation, sibling verification into a substitution lemma, inspection into elementary closure and cut subgraph extraction, and version evolution into proof reuse at unchanged interfaces. The central theorem states that a valid refinement DAG refines all admitted root intents, conditional on explicit executable–realization and profile–admissibility premises. The report develops the DAG definition, subgraph and cut extraction with frontier soundness, the refinement–DAG soundness proof, the cutoff theorem, a five–gate agent protocol, a four–step evolution loop, and a complete durable–acknowledgement example, and it relates the paper calculus to a checked Lean 4 mechanization kernel.

---

1. Introduction

Section titled “1. Introduction”

1.1 The problem

Section titled “1.1 The problem”

An agent is asked to add a feature or change a subsystem. It receives a requirement sentence and produces code. Between the sentence and the code lie decisions the sentence does not contain: data layout, ownership, error semantics, protocol ordering, retry policy, concurrency control, and edge–case handling. When the agent fills that gap in one leap, three failures follow.

Wide abstraction leap. The agent invents the missing design while writing the implementation. Implicit choices become hidden assumptions: a synchronous clock, a lossless channel, an atomic write that is not atomic, a lock that is not held across the operation that needs it. The resulting system can pass unit tests and still violate the requirement under interleaving, failure, or scale.

Rationale erasure and silent test weakening. A source repository records the final code, not the ordered argument for why a structure or protocol was chosen from upstream constraints, nor why the alternatives were rejected. A later agent cannot distinguish a load–bearing invariant from an incidental detail. Faced with a failing test, the cheapest repair is often to delete the assertion or weaken the condition. The build passes; the invariant is gone.

Human inscrutability. There is no trustworthy intermediate view. To know what the system does, a human must read thousands of lines of agent–generated code, reconstruct the design from the implementation, and guess which parts are intentional. Review degrades into rubber–stamping or line–by–line debugging.

1.2 Why post-hoc auditing does not address the problem

Section titled “1.2 Why post-hoc auditing does not address the problem”

An audit that runs after the code exists can detect some violations, but it does not prevent the leap that produced them. A diff algorithm can show what changed, but it cannot show which invariant the change was supposed to preserve. A test suite can encode a requirement, but a test is evidence about a sample, not a proof of a universal behavioral obligation, and it is exactly the artifact an agent can weaken. The problem is structural: the development process has no mandatory intermediate objects that are both behavioral and checkable.

1.3 The stratified thesis

Section titled “1.3 The stratified thesis”

Development proceeds through a finite Refinement DAG

$$G = \left(V,\ M,\ \chi,\ I,\ J,\ E\right)$$
G = (V , M, χ, I, J, E)

rooted at top-level system requirements and terminating at executable leaves. Here, “stratification” is the **partial-order acyclicity of the Refinement DAG** from root specifications down to executable leaves—not a rigid requirement that all subsystems share a fixed global layer count LL or pad shallower branches with dummy nodes.

At each cell $v \in V$v ∈ V , the developer records:

- the intents active at vv and the acyclic upstream support links that activate them;
- the context premises and design rationales for vv, with alternatives and reopening conditions;
- an explicit state machine M_vMv for vv;
- if vv is an internal cell ($\mathrm{ch}(v) \neq \varnothing$ch(v) = ∅), a decomposition into child contracts for its direct successors $\mathrm{ch}(v)$ch(v); if vv is a leaf ($\mathrm{ch}(v) = \varnothing$ch(v) = ∅), an executable program P_vPv;
- evidence that the cell’s machine satisfies its intents and that its decomposition or realization is sound.

A candidate advances only when the required checks at that cell and its outgoing edges succeed. Leaf cells ($V_{\text{leaf}} = \{\ell \in V \mid \mathrm{ch}(\ell) = \varnothing\}$ Vleaf = {ℓ ∈ V ∣ ch(ℓ) = ∅}) carry executable programs under declared execution semantics. Stringent development validity includes the operational realization of connector wiring and the selected profile’s initialization, input, and progress conditions; the soundness theorem is conditional on them. The result is a DAG of behavior inclusions, where every link is small enough to verify and every link is retained as evidence.

The prevention claim is precise. An implementation that violates a protected, formally expressed obligation cannot be accepted by a sound gate whose complete dependencies are preserved. The gate does not make the agent intelligent; it makes the agent’s obligations explicit, local, ordered, and checkable.

1.4 Contributions

Section titled “1.4 Contributions”

1. A refinement calculus in which intent satisfaction, child contract refinement, connector decomposition, and code realization are one relation: behavior inclusion under a typed edge abstraction map.
2. An acyclic topological support evaluation with a separate evidence stratum and a protected-obligation preservation rule, so neither a failed check nor a support edit silently erases the obligation.
3. A Refinement-DAG soundness theorem whose chain is leaf realization, connector monotonicity, child refinement, and inclusion transitivity, established by well-founded induction on the DAG.
4. Elementary subgraph and cross-section extraction on the DAG using three set primitives—predecessor closure, successor closure, and root-covering cuts—with pointwise and frontier soundness guarantees.
5. A sibling-independence theorem with explicit composition conditions, including interaction ownership and executable closure.
6. An incremental frame and interface cutoff theorem: a versioned evolution model where modifications, splits, merges, or refactorings reuse every unchanged obligation proposition by identity, requiring zero ancestor solver re-execution.

7. A five-gate development protocol and a four-step evolution loop, including a repair taxonomy that separates semantic counterexamples, failed proof witnesses, and unresolved checker results.
8. A complete durable-acknowledgement example, worked through a two-level Refinement DAG with a single-use interface precondition, a visible durability observation, leaf pseudocode, an agent error, its counterexample, and two repair paths.
9. A mechanization target whose kernel is checked in 718 lines of Lean 4.

1.5 Roadmap

Section titled "1.5 Roadmap"

Section 2 fixes the refinement algebra, typed edge abstractions, observation extension (including complexity bounds), and admissible input restrictions. Section 3 defines the Refinement DAG, support evaluation, context premises, validity, the DAG soundness theorem, and subgraph/cut extraction with frontier soundness. Section 4 details the verification obligations and evidence discipline. Section 5 treats decomposition, connectors, and sibling independence. Section 6 gives the development protocol. Section 7 gives versioned evolution, contextual intent projection, exact obligation propositions, the incremental frame theorem, and the evolution loop. Section 8 works the durable-acknowledgement example. Section 9 specifies the mechanization target and relates it to `JirlCalculus.lean`. Sections 10—11 cover related work and open problems. Appendices A—B give proof details and notation.

---

2. The Refinement Algebra

Section titled "2. The Refinement Algebra"

2.1 Observations, machines, and specifications

Section titled "2.1 Observations, machines, and specifications"

Fix an interface XX. An **observation space** O_XOX is a set of observations of XX. The core instance is the set of finite traces over the visible actions of XX; the liveness instance adds maximal traces under a declared fairness condition; the timed and probabilistic instances replace traces with timed or measure-valued observations. A **specification** at XX is a subset $\Phi \subseteq O_X$Φ ⊆ OX.

A **machine** at XX is a structure

$$M = (S, S_0, \Sigma, \to, \mathrm{obs}_M)$$
$$\mathsf{M} = (\mathsf{S}, \mathsf{S0}, \Sigma, \to, \mathsf{obsM})$$

where SS is a state set, $S_0 \subseteq S$S0 ⊆ S the initial states, ΣΣ a partitioned alphabet of inputs, outputs, and internal actions, $\to \subseteq S \times \Sigma \times S$ →⊆ S × Σ × S the transition relation, and obs_MobsM maps executions to observations. Its **behavior** is

$$\mathrm{beh}(M) = \{\mathrm{obs}_M(\rho) \mid \rho \text{ is an execution of } M\} \subseteq O_X.$$
$$\mathsf{beh}(\mathsf{M}) = \{\mathsf{obsM}(\rho) \mid \rho \text{ is an execution of } \mathsf{M}\} \subseteq \mathsf{OX}.$$

For the trace kernel, $\mathrm{beh}(M)$beh(M) is a trace set and is written $\mathrm{tr}(M)$tr(M).

2.2 Satisfaction, refinement, and typed edge abstraction

Section titled "2.2 Satisfaction, refinement, and typed edge abstraction"

Satisfaction and **refinement** are inclusion:

$$M \vDash \Phi \iff \mathrm{beh}(M) \subseteq \Phi, \qquad M \preceq A \iff \mathrm{beh}(M) \subseteq \mathrm{beh}(A).$$
$$\mathsf{M} \vDash \Phi \iff \mathsf{beh}(\mathsf{M}) \subseteq \Phi, \qquad \mathsf{M} \preceq \mathsf{A} \iff \mathsf{beh}(\mathsf{M}) \subseteq \mathsf{beh}(\mathsf{A}).$$

For every decomposition edge (v, c)(v, c) where $c \in \mathrm{ch}(v)$c ∈ ch(v), we fix the typing:

$$\alpha_{v,c}\colon O_{X_c} \longrightarrow O_{X_{v,c}}, \qquad \mathrm{beh}(B_{v,c}) \subseteq O_{X_{v,c}}, \qquad K_v\colon \prod_{c \,\in\, \mathrm{ch}(v)} P(O_{X_{v,c}}) \longrightarrow P(O_{X_v}).$$

αv,c : OXc ⟶ OXv,c, beh(Bv,c) ⊆ OXv,c, Kv : c∈ch(v)∏P(OXv,c) ⟶ P(OXv).

Here, X_cXc is the child cell's interface, $X_{v,c}$Xv,c is the slot interface assigned to child cc by parent vv, and $B_{v,c}$Bv,c is the contract for slot (v,c)(v, c) at interface $X_{v,c}$Xv,c. Under the abstraction map $\alpha_{v,c}$αv,c, child refinement is:

$$M_c \preceq_{\alpha_{v,c}} B_{v,c} \iff \alpha_{v,c}(\mathrm{beh}(M_c)) \subseteq \mathrm{beh}(B_{v,c}) \iff \mathrm{beh}(M_c) \subseteq \alpha_{v,c}^{*}\big(\mathrm{beh}(B_{v,c})\big),$$

Mc ≼αv,c Bv,c ⟺ αv,c(beh(Mc)) ⊆ beh(Bv,c) ⟺ beh(Mc) ⊆ αv,c∗(beh(Bv,c)),

where $\alpha_{v,c}^{*}(\Phi) = \{o \in O_{X_c} \mid \alpha_{v,c}(o) \in \Phi\}$αv,c∗(Φ) = {o ∈ OXc ∣ αv,c(o) ∈ Φ}. Inclusion is reflexive and transitive, and inverse image is functorial: $(\alpha_2 \circ \alpha_1)^{*} = \alpha_1^{*} \circ \alpha_2^{*}$(α2 ∘ α1)∗ = α1∗ ∘ α2∗ and $\mathrm{id}^{*} = \mathrm{id}$id∗ = id.

This typing cleanly separates the role of the child from the role of the parent: a child's behavior $\mathrm{beh}(M_c)$beh(Mc) refines its assigned slot contract $B_{v,c}$Bv,c under $\alpha_{v,c}$αv,c, whereas the parent's behavior $\mathrm{beh}(M_v)$beh(Mv) is refined by the connector image $K_v(\{\mathrm{beh}(B_{v,c})\}_c)$Kv({beh(Bv,c)}c).

Observation extension & emergent child properties (e.g., complexity bounds). The single inclusion relation $\subseteq$ ⊆ handles the case where a parent does not constrain complexity while a child introduces a high-performance complexity bound. When the child's observation space O_{X_c}OXc extends observations with resource or cost dimensions $(\tau, \mathrm{size}, \mathrm{cost})$(τ, size, cost) that $\alpha_{v,c}$αv,c projects out to $\tau \in O_{X_{v,c}}$ τ ∈ OXv,c, satisfying both the parent slot contract $B_{v,c}$Bv,c and the child's local complexity specification $\Phi_c^{\mathrm{cost}} = \{o \in O_{X_c} \mid \mathrm{cost}(o) \le f(\mathrm{size}(o))\}$Φccost = {o ∈ OXc ∣ cost(o) ⩽ f(size(o))} is a single pullback-intersection inclusion:

$$\mathrm{beh}(M_c) \;\subseteq\; \alpha_{v,c}^{*}\big(\mathrm{beh}(B_{v,c})\big) \;\cap\; \Phi_c^{\mathrm{cost}}.$$

beh(Mc) ⊆ αv,c∗(beh(Bv,c)) ∩ Φccost.

If $O_{X_{v,c}}$OXv,c already includes an unconstrained cost coordinate $\mathrm{cost} \in [0, \infty)$cost ∈ [0, ∞), adding $\mathrm{cost} \le f(\mathrm{size})$cost ⩽ f(size) simply narrows the allowed behavior set, which is directly $\subseteq$ ⊆. Note that exporting a whole-parent complexity bound requires a cost-preserving observation space at vv and a cost-composition law on K_vKv.

General-purpose children & admissible input histories. When a reusable child component accepts a wider input domain than a particular parent sends, let $A_{v,c}$Av,c be the set of input histories guaranteed by the parent's connector W_vWv, and write $\mathrm{beh}(M_c)\restriction_{A_{v,c}}$ beh(Mc) ↾Av,c for the behaviors generated by executions of M_cMc whose input histories lie in $A_{v,c}$Av,c. The edge refinement obligation is:

$$\alpha_{v,c}\big(\mathrm{beh}(M_c)\restriction_{A_{v,c}}\big) \;\subseteq\; \mathrm{beh}(B_{v,c}),$$

αv,c(beh(Mc) ↾Av,c) ⊆ beh(Bv,c),

accompanied by input receptiveness and progress preservation on $A_{v,c}$Av,c.

2.3 Connectors

Section titled "2.3 Connectors"

A **connector** at a parent cell vv is a map on behavior sets $K_v\colon \prod_{c \,\in\, \mathrm{ch}(v)} P(O_{X_{v,c}}) \longrightarrow P(O_{X_v})$Kv : Πc∈ch(v) P(OXv,c) ⟶ P(OXv), and it is **admissible** when it is monotone:

$$B_{v,c} \subseteq B'_{v,c} \text{ for all } c \in \mathrm{ch}(v) \implies K_v(\{B_{v,c}\}_c) \subseteq K_v(\{B'_{v,c}\}_c).$$

Bv,c ⊆ Bv,c′ for all c ∈ ch(v) ⟹ Kv({Bv,c}c) ⊆ Kv({Bv,c′}c).

The operational reading is a wiring skeleton W_vWv that routes, synchronizes, interleaves, and hides child actions, with no state of its own that is not visible in the children or in a declared substrate primitive. A **trajectory model** of W_vWv supplies:

- a set legal_{W_v}legalWv of legal shared trajectories rr;
- for each child cc, a slice $\mathrm{slice}_c(r)$slicec(r) in $O_{X_{v,c}}$OXv,c;
- a projection $\mathrm{obs}_v(r)$obsv(r) in O_{X_v}OXv.

The induced transformer is

$$K_v(\{B_{v,c}\}_c) = \left\{ \mathrm{obs}_v(r) \;\middle|\; \mathrm{legal}_{W_v}(r) \wedge \forall c \in \mathrm{ch}(v).\ \mathrm{slice}_c(r) \in B_{v,c} \right\}.$$

Kv({Bv,c}c) = { obsv(r) | legalWv(r) ∧ ∀c ∈ ch(v). slicec(r) ∈ Bv,c }.

Lemma 1 (Trajectory connectors are monotone). If $B_{v,c} \subseteq B'_{v,c}$Bv,c ⊆ Bv,c′ for every $c \in \mathrm{ch}(v)$c ∈ ch(v), then $K_v(\{B_{v,c}\}_c) \subseteq K_v(\{B'_{v,c}\}_c)$Kv({Bv,c}c) ⊆ Kv({Bv,c′}c).

Proof. Take $o \in K_v(\{B_{v,c}\}_c)$o ∈ Kv({Bv,c}c). There is a legal rr with $\mathrm{obs}_v(r) = o$obsv(r) = o and $\mathrm{slice}_c(r) \in B_{v,c}$slicec(r) ∈ Bv,c for every c c. Since $B_{v,c} \subseteq B'_{v,c}$Bv,c ⊆ Bv,c′, each slice lies in $B'_{v,c}$Bv,c′ as well, so $o \in K_v(\{B'_{v,c}\}_c)$o ∈ Kv({Bv,c′}c). ■■

The trajectory model is the mathematical content of the wiring. Its **operational soundness** obligation is that the implementation of W_vWv produces only observations in the transformer's output. This is where routing, synchronization, initialization, and shared-resource behavior are established. It is a property of the connector, discharged once per wiring pattern, not a per-child obligation.

2.4 The single relation and its five uses

Section titled "2.4 The single relation and its five uses"

Every behavioral obligation in the calculus is an instance of inclusion:

Use	Left side	Right side	Where checked
Satisfaction	$\mathrm{beh}(M_v)$beh(Mv)	active intent $\Phi(v)$Φ(v)	cell vv
Wiring adequacy	$\mathrm{Beh}(C_v[Q_c])$Beh(Cv[Qc])	$K_v(\{\alpha_{v,c}(Q_c)\}_c)$Kv({αv,c(Qc)}c)	decomposed cell vv, universally over admissible Q_cQc
Decomposition	$K_v(\{\mathrm{beh}(B_{v,c})\}_c)$Kv({beh(Bv,c)}c)	$\mathrm{beh}(M_v)$beh(Mv)	decomposed cell vv
Refinement	$\alpha_{v,c}(\mathrm{beh}(M_c))$αv,c(beh(Mc))	contract $\mathrm{beh}(B_{v,c})$beh(Bv,c)	decomposition edge (v,c)(v, c)
Realization	$\mathrm{beh}(P_\ell)$beh(Pℓ)	$\mathrm{beh}(M_\ell)$beh(Mℓ)	leaf cell $\ell \in V_{\mathrm{leaf}}$ℓ ∈ Vleaf

The algebra is small on purpose. The paper's correctness argument is the transitive closure of this table, and the engineering content is the discipline that decides what to put on each side.

2.5 Semantic profiles

Section titled "2.5 Semantic profiles"

A **semantic profile** ΓΓ fixes the observation class and the obligations that accompany it:

- **Safety profile.** Observations are finite visible traces; $M \preceq A$M ≤ A is trace inclusion.
- **Liveness profile.** Observations are maximal traces under a declared fairness condition; refinement preserves maximality and fairness, and a progress obligation accompanies the trace inclusion.
- **Timed and probabilistic profiles.** Observations are timed traces or path measures; refinement uses the corresponding measurable or timed preorder.

Two side conditions belong to every open-interface profile:

- **Input receptiveness.** If a contract permits an input at a related state, the child must accept it. Without this condition, a child can satisfy a safety contract by refusing every request, which is not an admissible replacement.
- **Progress preservation.** For a liveness contract, the child must preserve the required maximal fair executions. Finite-prefix inclusion alone admits a machine that accepts a request and then stops.

The core theorems are stated for the trace profile with these two side conditions. The timed and probabilistic instantiations use the same graph structure and the same schema; only the profile's refinement relation and composition lemma change.

2.6 Boundary discipline

Section titled "2.6 Boundary discipline"

A contract constrains observable behavior at the child interface and the child's duties to the connector. It may specify:

- the inputs the child must accept;
- the outputs it may produce and their order;
- relations between its outputs and shared connector state;
- progress or termination duties;
- the environment assumptions under which those duties hold.

It does not prescribe the child's internal algorithm, data layout, module structure, or implementation strategy. The child is free to choose any implementation whose behavior lies within the contract. This is the boundary rule that makes independent development possible: **the parent owns the interface; the child owns the implementation.**

---

3. The Refinement DAG and Soundness

Section titled "3. The Refinement DAG and Soundness"

3.1 The development DAG record

Section titled "3.1 The development DAG record"

A **development record** is a finite tuple

$$G = \big(V,\ M,\ \chi,\ I,\ J,\ E\big).$$
G = (V , M, χ, I, J, E).

The fields define the modular and behavioral boundary of the development without layer subscripts:

VV **— cells.** A finite set of components.

MM **— local models.** For each cell $v \in V$v ∈ V , its local specification and machine record:

$$M(v) = \big(O_{X_v},\ \Phi(v),\ M_v,\ \Gamma_v\big),$$
M(v) = (OXv , Φ(v), Mv , Γv),

where O_{X_v}OXv is the observation space at interface X_vXv, M_vMv is its state machine, $\Phi(v) = \bigcap_{c \in A(v)} [\![c]\!]$Φ(v) = ∩c∈A(v)[[c]] is the semantic conjunction of active intents owned by vv, and Γ_vΓv records the semantic profile and local context premises ΔP_vΔPv (domain axioms and environment assumptions).

χχ **— decomposition or realization.** For an internal cell $(\mathrm{ch}(v) \neq \varnothing$ch(v) $= \varnothing)$, $\chi(v)$χ(v) is a typed decomposition record:

$$\chi(v) = \big(W_v,\ \mathrm{ch}(v),\ (B_{v,c},\ \alpha_{v,c})_{c \in \mathrm{ch}(v)}\big),$$
χ(v) = (Wv , ch(v), (Bv,c, αv,c)c∈ch(v)),

where W_vWv is a wiring skeleton, $\mathrm{ch}(v) \subset V$ch(v) ⊂ V is a finite set of child cells, $B_{v,c}$Bv,c is the contract assigned to child cc at slot interface $X_{v,c}$Xv,c, and $\alpha_{v,c}: O_{X_c} \to O_{X_{v,c}}$αv,c : OXc → OXv,c is the typed edge abstraction map. For a leaf cell ℓℓ ($\mathrm{ch}(\ell) = \varnothing$ch(ℓ) = ∅), $\chi(\ell) = P_\ell$χ(ℓ) = Pℓ is the executable leaf program under the declared runtime semantics.

The child relation $v \to c \iff c \in \mathrm{ch}(v)$v → c ⟺ c ∈ ch(v) forms a finite **directed acyclic graph (DAG)** with roots $V_{\mathrm{root}} = \{r \in V \mid \mathrm{par}(r) = \varnothing\}$Vroot = {r ∈ V ∣ par(r) = ∅} and leaves $V_{\mathrm{leaf}} = \{\ell \in V \mid \mathrm{ch}(\ell) = \varnothing\}$Vleaf = {ℓ ∈ V ∣ ch(ℓ) = ∅}, where $\mathrm{par}(c) = \{v \in V \mid c \in \mathrm{ch}(v)\}$par(c) = {v ∈ V ∣ c ∈ ch(v)}. No dummy padding nodes or uniform leaf depths are needed.

II — intent records. A finite set of intents. Each $c \in I$c ∈ I carries:

- a stable identifier $\mathrm{id}(c)$id(c);
- an owner cell $\mathrm{owner}(c) \in V$owner(c) ∈ V;
- a predicate $[\![c]\!] \subseteq O_{X_{\mathrm{owner}(c)}}$[[c]] ⊆ OXowner(c);
- an authority tag: rootroot, $\mathrm{protected}$protected, or $\mathrm{delegated}$delegated.

Ownership is total and unique: every active intent is owned by exactly one cell, so $\bigcup_{v \in V} A(v) = A$⋃v∈V A(v) = A and $A(u) \cap A(v) = \varnothing$ A(u) ∩ A(v) = ∅ for $u \neq v$u ≠ v.

JJ — support links. For each $c \in I$c ∈ I, a finite set $J(c)$J(c) of support groups. Each group $P \in J(c)$P ∈ J(c) is a finite conjunctive set of upstream intent references $P \subseteq I$P ⊆ I, forming an acyclic dependency relation on II. Multiple groups represent disjunctive alternatives.

EE — evidence map. A finite map from required local behavioral-inclusion propositions to evidence records. The evidence stratum is separate from the intent stratum: an intent's activation does not depend on whether its evidence exists or passes.

3.2 Active intents and topological support evaluation

Section titled "3.2 Active intents and topological support evaluation"

The **active intents** $A \subseteq I$A ⊆ I are defined as the least fixed point:

$$c \in A \iff c \in I^{\mathrm{root}} \vee \exists P \in J(c).\ P \subseteq A.$$
c ∈ A ⟺ c ∈ Iroot ∨ ∃P ∈ J(c). P ⊆ A.

Because the support relation JJ is acyclic on the finite set II, AA is evaluated deterministically in a single forward pass along any topological order of (I, J)(I, J) without circularity.

Lemma 2 (Evidence separation). AA is a function of (I, J)(I, J) only and does not mention EE. Consequently, a failed, missing, or pending evidence record leaves every obligation it concerns active and blocking.

Proof. The fixed-point condition defining AA refers only to intent records and support links. The evidence stratum EE does not occur in it. ■■

3.3 Context premises, design provenance, and staleness

Section titled "3.3 Context premises, design provenance, and staleness"

Architectural choices—including contract declaration, connector selection, and component decomposition—are recorded at cell boundaries. For each cell $v \in V$v ∈ V, its local context premises ΔP_vΔPv form a finite set of references to upstream intents, parent contracts, and environmental assumptions on which vv's model and decomposition depend, with transitive upstream closure ΔP_v^*ΔPv∗. Superseded or alternative design options are preserved as historical snapshots in the version sequence $\{G_t\}_{t \in N}$Gt t∈N.

If an upstream intent or context premise xx changes, downstream components in its dependency closure become stale:

$$\mathrm{stale}(x) = \{v \in V \mid x \in \Delta P_v^*\}.$$
stale(x) = {v ∈ V | x ∈ ΔPv∗}.

Because prerequisites are strictly upstream, the reference graph is acyclic and the stale set is computed in linear time. A stale premise blocks publication of every cell that depends on it until re-warranted or replaced.

3.4 Intra-DAG validity

Section titled "3.4 Intra-DAG validity"

For a leaf $\ell \in V_{\text{leaf}}$ℓ ∈ Vleaf, $\mathrm{Beh}_{\text{exec}}(\ell) = \mathrm{beh}(P_\ell)$Behexec(ℓ) = beh(Pℓ) under the declared execution semantics. For a decomposed cell vv, concrete wiring C_vCv runs with the executable children to produce $\mathrm{Beh}_{\text{exec}}(v)$Behexec(v). The connector transformer K_vKv is the abstract specification image of C_vCv.

A development DAG $G = (V, M, \chi, I, J, E)$G = (V, M, χ, I, J, E) is **valid** when conditions 1–6 hold for GG (and condition 7 holds when publishing GG as a versioned update to an existing development):

1. **Support well-formedness and ownership coverage.** Every active intent in AA has an active support group (or is root); support links are acyclic; and $\bigcup_{v \in V} A(v) = A$∪v∈V A(v) = A with each intent uniquely owned.
2. **Context premise validity.** For every cell $v \in V$v ∈ V, local premises ΔP_vΔPv are warranted and compatible with upstream cells.
3. **Satisfaction.** For every cell $v \in V$v ∈ V and active intent $c \in A(v)$c ∈ A(v), evidence EE contains a valid record for

$$\mathrm{beh}(M_v) \subseteq [\![c]\!].$$
beh(Mv) ⊆ [[c]].

4. **Decomposition.** For every non-leaf cell $v \in V \setminus V_{\text{leaf}}$v ∈ V ∖ Vleaf, evidence EE contains a valid record for

$$K_v\left(\{\mathrm{beh}(B_{v,c})\}_{c \in \mathrm{ch}(v)}\right) \subseteq \mathrm{beh}(M_v).$$
Kv({beh(Bv,c)}c∈ch(v)) ⊆ beh(Mv).

5. **Executable realization.** For a decomposed cell vv, evidence EE contains a **wiring adequacy certificate** for C_vCv:

$$\forall (Q_c)_{c \in \mathrm{ch}(v)}.\ \left(\forall c.\ Q_c \text{ is admissible for } X_{v,c}, \Gamma_c\right) \implies \mathrm{Beh}(C_v[Q_c]) \subseteq K_v\left(\{\alpha_{v,c}(Q_c)\}_c\right).$$
∀(Qc)c∈ch(v). (∀c. Qc is admissible for Xv,c, Γc) ⟹ Beh(Cv[Qc]) ⊆ Kv({αv,c(Qc)}c).

   The operational instance $\mathrm{Beh}_{\text{exec}}(v) \subseteq K_v(\{\alpha_{v,c}(\mathrm{Beh}_{\text{exec}}(c))\}_c)$Behexec(v) ⊆ Kv({αv,c(Behexec(c))}c) follows by application to executable child behaviors. For a leaf $\ell \in V_{\text{leaf}}$ℓ ∈ Vleaf, evidence EE certifies $\mathrm{Beh}_{\text{exec}}(\ell) = \mathrm{beh}(P_\ell) \subseteq \mathrm{beh}(M_\ell)$Behexec(ℓ) = beh(Pℓ) ⊆ beh(Mℓ), with a nonempty initial state set.
6. **Profile admissibility.** The selected semantic profile's conditions hold:
   - *Initialization.* Initial states are within the invariant, and initial state sets are nonempty.
   - *Input receptiveness.* If a contract permits an input at a related state, the child accepts it.
   - *Progress preservation.* For liveness claims, required maximal fair executions are preserved.
7. **Protected obligation preservation.** Every protected obligation active in the previous version remains active with identical semantic content, unless an authorized disposition records its retirement or weakening (§3.8).

Conditions 1–6 define the intra-DAG validity of a development record GG, and are the hypotheses consumed by the soundness theorem (Theorem 1). Condition 7 is the inter-version preservation condition consumed by the publication gate to prevent silent obligation erasure.

3.5 Edge refinement and the shared-child discipline

Section titled "3.5 Edge refinement and the shared-child discipline"

For every decomposition edge (v, c)(v, c) with $c \in \mathrm{ch}(v)$c ∈ ch(v), evidence EE contains a valid record for edge refinement:

$$\alpha_{v,c}\big(\mathrm{beh}(M_c)\big) \subseteq \mathrm{beh}(B_{v,c}) \iff \mathrm{beh}(M_c) \subseteq \alpha^{*}_{v,c}\big(\mathrm{beh}(B_{v,c})\big).$$
$$\text{αv,c(beh(Mc))} \subseteq \text{beh(Bv,c)} \iff \text{beh(Mc)} \subseteq \text{αv,c}*\text{(beh(Bv,c))}.$$

The edge refinement compares a child implementation with the contract its parent declared, under the edge abstraction map $\alpha_{v,c}$αv,c. It does not compare the child with an obsolete implementation; the contract is the complete interface.

When a child cell cc has multiple parents $\mathrm{par}(c) = \{u_1, \ldots, u_m\}$par(c) = {u1, … , um} (a shared child in the DAG), two disciplines govern the shared node:

1. **Edge-indexed refinement.** The child machine M_cMc must simultaneously satisfy each incoming slot contract:

$$\forall u \in \mathrm{par}(c), \quad \alpha_{u,c}\big(\mathrm{beh}(M_c)\big) \subseteq \mathrm{beh}(B_{u,c}) \iff \mathrm{beh}(M_c) \subseteq \bigcap_{u \in \mathrm{par}(c)} \alpha^{*}_{u,c}\big(\mathrm{beh}(B_{u,c})\big).$$
$$\forall \text{u} \in \text{par(c)}, \quad \text{αu,c(beh(Mc))} \subseteq \text{beh(Bu,c)} \iff \text{beh(Mc)} \subseteq \text{u∈par(c)∩ αu,c}*\text{(beh(Bu,c))}.$$

2. **Instance vs. singleton scope.** If each parent instantiates a separate runtime copy of cc (or cc is purely stateless), edge-indexed refinement suffices. If parents share a single stateful runtime instance of cc, $\mathrm{Beh}_{\mathrm{exec}}(c)$Behexec(c) and M_cMc must range over the **aggregate multi-client history** owned at a selected common dominating ancestor boundary. The two-sentence shared-counter witness illustrates why: two clients that each expect `1` on their first call in isolation observe `1` and `2` when sharing one counter instance. Thus, separate isolated single-client contracts do not compose without accounting for multi-client interleavings at the dominating ancestor.

3.6 Refinement-DAG soundness

Section titled “3.6 Refinement-DAG soundness”

Theorem 1 (Refinement-DAG soundness). Let $G = (V, M, \chi, I, J, E)$G = (V , M, χ, I , J , E) be a Refinement DAG. Suppose:

- (A1) GG satisfies the validity conditions 1–6 of §3.4;
- (A2) every edge (v, c)(v, c) with $c \in \mathrm{ch}(v)$c ∈ ch(v) satisfies edge refinement (§3.5);
- (A3) the edge abstraction maps are typed and compose along paths in the DAG.

Then for every cell $v \in V$v ∈ V , $\mathrm{Beh}_{\mathrm{exec}}(v) \subseteq \mathrm{beh}(M_v)$Behexec(v) ⊆ beh(Mv), and for every active intent $c \in A(v)$c ∈ A(v), $\mathrm{Beh}_{\mathrm{exec}}(v) \subseteq [\![c]\!]$Behexec(v) ⊆ ⟦c⟧. In particular, for every root $r \in V_{\mathrm{root}}$r ∈ Vroot,

$$\mathrm{Beh}_{\mathrm{exec}}(r) \subseteq \Phi(r) = \bigcap_{c \in A(r)} [\![c]\!],$$
$$\text{Behexec(r)} \subseteq \Phi\text{(r)} = \text{c∈A(r)∩⟦c⟧},$$

the semantic conjunction at root rr, not the syntactic set $A(r)$A(r).

Proof. By well-founded induction on the finite acyclic child relation $v \to c$v → c on VV .

Base case ($\ell \in V_{\mathrm{leaf}}$ℓ ∈ Vleaf, $\mathrm{ch}(\ell) = \varnothing$ch(ℓ) = ∅). Condition 5 gives $\mathrm{Beh}_{\mathrm{exec}}(\ell) = \mathrm{beh}(P_\ell) \subseteq \mathrm{beh}(M_\ell)$Behexec(ℓ) = beh(Pℓ) ⊆ beh(Mℓ) and the nonempty initial state condition. Condition 3 gives $\mathrm{beh}(M_\ell) \subseteq [\![c]\!]$beh(Mℓ) ⊆ ⟦c⟧ for every active intent $c \in A(\ell)$c ∈ A(ℓ).

Inductive step ($v \notin V_{\mathrm{leaf}}$v ∉ Vleaf). Assume the claim for every child $c \in \mathrm{ch}(v)$c ∈ ch(v), and write $B_c = \mathrm{Beh}_{\mathrm{exec}}(c)$Bc = Behexec(c). Let $B_{v,c}$Bv,c be the contract declared by vv for cc. Condition 5 gives the universal wiring-adequacy certificate; instantiating it at $Q_c = B_c$Qc = Bc, which is admissible by condition 6, gives the operational inclusion

$$\mathrm{Beh}_{\mathrm{exec}}(v) \subseteq K_v\left(\{\alpha_{v,c}(B_c)\}_{c \in \mathrm{ch}(v)}\right). \quad (1)$$
Behexec(v) ⊆ Kv({αv,c(Bc)}c∈ch(v)). (1)

The induction hypothesis gives $B_c \subseteq \mathrm{beh}(M_c)$Bc ⊆ beh(Mc), hence $\alpha_{v,c}(B_c) \subseteq \alpha_{v,c}(\mathrm{beh}(M_c))$αv,c(Bc) ⊆ αv,c(beh(Mc)). Monotonicity of K_v Kv (Lemma 1) gives

$$K_v\left(\{\alpha_{v,c}(B_c)\}_c\right) \subseteq K_v\left(\{\alpha_{v,c}(\mathrm{beh}(M_c))\}_c\right). \quad (2)$$
Kv({αv,c(Bc)}c) ⊆ Kv({αv,c(beh(Mc))}c). (2)

Edge refinement (A2) gives $\alpha_{v,c}(\mathrm{beh}(M_c)) \subseteq \mathrm{beh}(B_{v,c})$αv,c(beh(Mc)) ⊆ beh(Bv,c). Monotonicity gives

$$K_v\left(\{\alpha_{v,c}(\mathrm{beh}(M_c))\}_c\right) \subseteq K_v\left(\{\mathrm{beh}(B_{v,c})\}_c\right). \quad (3)$$
Kv({αv,c(beh(Mc))}c) ⊆ Kv({beh(Bv,c)}c). (3)

Condition 4 gives

$$K_v\left(\{\mathrm{beh}(B_{v,c})\}_c\right) \subseteq \mathrm{beh}(M_v). \quad (4)$$
Kv({beh(Bv,c)}c) ⊆ beh(Mv). (4)

Chaining (1)–(4) yields $\mathrm{Beh}_{\mathrm{exec}}(v) \subseteq \mathrm{beh}(M_v)$Behexec(v) ⊆ beh(Mv). Condition 3 gives $\mathrm{beh}(M_v) \subseteq [\![c]\!]\mathrm{beh}$(Mv) ⊆ ⟦c⟧ for every active intent $c \in A(v)$c ∈ A(v). Condition 6 supplies the profile's initialization, input, and progress conditions, so no step uses an empty behavior or a refused input to satisfy an obligation. For every root $r \in V_{\mathrm{root}}$r ∈ Vroot, this yields $\mathrm{Beh}_{\mathrm{exec}}(r) \subseteq \Phi(r)$Behexec(r) ⊆ Φ(r). ■■

The proof is the spine of the calculus. It uses executable realization, monotonicity, child refinement, decomposition, and transitivity. Stratification enters in the well-founded induction: every child's proof is a separate premise, and the parent's decomposition is checked against contracts rather than against the concrete product of child states. The operational counterexample — identity KK, child $\{ok\}${ok}, parent $\{ok\}${ok}, actual wiring $\{bad\}${bad} — is excluded by condition 5, because $\{bad\} \subseteq \{ok\}${bad} ⊆ {ok} is false. The empty-initial-state counterexample is excluded by condition 6.

3.7 Subgraph extraction, cuts, and frontier soundness

Section titled "3.7 Subgraph extraction, cuts, and frontier soundness"

Human inspection and agent context scoping are formalized as **information extraction on the finite DAG** (V, ch)**(V , ch)** using **only three elementary set primitives and their intersections**:

1. **Predecessor (Ancestor) Closure.**

   $$\mathrm{Anc}(S) = \{u \in V \mid \exists x \in S,\ u \to^* x\}.$$
   Anc(S) = {u ∈ V ∣ ∃x ∈ S, u →∗ x}.

   This extracts the upstream provenance cone, answering *why* SS exists and what requirements govern it.
2. **Successor (Descendant) Closure.**

   $$\mathrm{Desc}(S) = \{w \in V \mid \exists x \in S,\ x \to^* w\},$$
   Desc(S) = {w ∈ V ∣ ∃x ∈ S, x →∗ w},

   along with immediate direct successors $\mathrm{ch}(v) = \{c \in V \mid v \to c\}$ch(v) = {c ∈ V ∣ v → c}. While $\mathrm{Desc}(S)$Desc(S) forms the reflexive transitive closure, $\mathrm{ch}(v)$ch(v) extracts the immediate outgoing edges of vv, providing the single-step boundary for local decomposition.

3. **Root–Covering Cut (Frontier).** An antichain $C \subseteq V$C ⊆ V that intersects every directed path from V_{root}Vroot to V_{leaf}Vleaf. Given a chosen focus antichain $K \subseteq V$K ⊆ V contained in a root–covering cut $C \supseteq K$C ⊇ K, the cut CC extracts a complete horizontal cross–section passing through KK.

All composite inspection views are simple set intersections (∩ ∩):

- **Connecting Interval between uu and ww:** $[u, w] = \text{Desc}(\{u\}) \cap \text{Anc}(\{w\})$[u, w] = Desc({u}) ∩ Anc({w});
- **Single–Step Decomposition View at internal cell $v \in V \setminus V_{\text{leaf}}$v ∈ V ∖ Vleaf:** $\{v\} \cup \text{ch}(v) = \text{Desc}(\{v\}) \cap \text{Anc}(\text{ch}(v))${v} ∪ ch(v) = Desc({v}) ∩ Anc(ch(v)) (and for a leaf $\ell \in V_{\text{leaf}}$ℓ ∈ Vleaf, $\text{ch}(\ell) = \varnothing$ch(ℓ) = ∅ and the inspection view is simply $\{\ell\}${ℓ});
- **Bounded Slab between Two Cuts C_1, C_2C1, C2:** $\text{Slice}(C_1, C_2) = \text{Desc}(C_1) \cap \text{Anc}(C_2)$Slice(C1, C2) = Desc(C1) ∩ Anc(C2) (and for $C_1 = V_{\text{root}}, C_2 = C$C1 = Vroot, C2 = C, the upper system truncated at cut CC).

Because architectural DAGs have modest size, all closure and cut extractions are evaluated directly by their set definitions without asymptotic complexity machinery.

Corollary 1 (Pointwise and Frontier Soundness).

- *Pointwise soundness.* For every cell $v \in V$v ∈ V, $\text{Beh}_{\text{exec}}(v) \subseteq \text{beh}(M_v) \subseteq \Phi(v)$Behexec(v) ⊆ beh(Mv) ⊆ Φ(v).
- *Frontier soundness.* For any root–covering cut CC, define the cut–abstracted behavior $H_C(u) = \text{beh}(M_u)$HC(u) = beh(Mu) for $u \in C$ u ∈ C and $H_C(v) = K_v(\{\alpha_{v,c}(H_C(c))\}_{c \in \text{ch}(v)})$HC(v) = Kv({αv,c(HC(c))}c∈ch(v)) for $v \in \text{Anc}(C) \setminus C$v ∈ Anc(C) ∖ C. If the upper region $\text{Anc}(C)$Anc(C) is valid modulo CC and $\text{Beh}_{\text{exec}}(u) \subseteq \text{beh}(M_u)$Behexec(u) ⊆ beh(Mu) holds for all $u \in C$u ∈ C, then for every root $r \in V_{\text{root}}$r ∈ Vroot,

$$\text{Beh}_{\text{exec}}(r) \subseteq H_C(r) \subseteq \text{beh}(M_r) \subseteq \Phi(r).$$
Behexec(r) ⊆ HC(r) ⊆ beh(Mr) ⊆ Φ(r).

The inspection direction is: every executable behavior maps under the abstractions to a behavior permitted by the cut's machines, and every universal obligation displayed at cut CC transfers to the code. The converse does not hold: an abstract cut model may admit behaviors that a concrete realization narrows. The cut view is an abstraction, not an equivalence.

3.8 Protected obligation preservation

Section titled "3.8 Protected obligation preservation"

Define the **effective protected inventory** of a version GG:

$$\text{Prot}(G) = \left\{ (\text{id}(c),\ [\![\, c \,]\!],\ \text{owner}(c)) : c \text{ is active in } G \text{ and } \text{authority}(c) \in \{\text{root}, \text{protected}\} \right\}.$$
Prot(G) = {(id(c), [[c]], owner(c)) : c is active in G and authority(c) ∈ {root, protected}}.

A candidate G'G′ **preserves** GG when for every $p \in \text{Prot}(G)$p ∈ Prot(G), either pp is active in G'G′ with the same semantic content, or an authorized disposition $d \in A_{G'}$d ∈ AG′ (§7.1) records pp, the authorization, and the rationale. A support edit, ownership change, or interpretation change that removes pp without such a disposition is rejected by validity condition 7. Ownership is total, so $\text{Prot}(G)$Prot(G) and the per–cell satisfaction sets range over the same active obligations; a protected interaction intent is assigned to the cell that owns the interaction boundary and is checked there.

The support–only witness: a protected intent cc has $J(c) = \{\{s\}\}$J(c) = {{s}} and ss is accepted. The candidate changes only $J(c)$J(c) to $\varnothing$∅. The identifier, predicate, and authority tag are unchanged, but AA loses cc, so its satisfaction evidence is no longer requested. Condition 7 rejects the candidate; only an authorized disposition can retire cc. Evidence failure does not deactivate an intent (Lemma 2), and a support or ownership edit does not deactivate a protected intent without a disposition (condition 7).

---

4. Verification Obligations

Section titled “4. Verification Obligations”

4.1 Satisfaction

Section titled “4.1 Satisfaction”

For a finite machine M_vMv and a specification ΦΦ, satisfaction is checked by exhaustive state-space exploration or by a certified invariant. For an infinite machine, it is checked by a theorem, a certified abstraction, or a bounded argument whose scope is recorded. The obligation is exactly $\mathrm{beh}(M_v) \subseteq \Phi(v)$beh(Mv) ⊆ Φ(v). A test is not, by itself, evidence for this universal inclusion. It is evidence for a bounded claim: no violation in the tested sample, or no violation within a bound. The evidence record must state the claim; the gate checks that the claim covers the obligation. A universal obligation is not discharged by a sampled test. A bounded intent can be discharged by a bounded check, provided the intent is stated with the same bound.

4.2 Decomposition and connector admissibility

Section titled “4.2 Decomposition and connector admissibility”

The decomposition obligation is

$$K_v\left(\{\mathrm{beh}(B_{v,c})\}_{c \in \mathrm{ch}(v)}\right) \subseteq \mathrm{beh}(M_v).$$
Kv({beh(Bv,c)}c∈ch(v)) ⊆ beh(Mv).

It is checked at the parent cell against abstract child contracts, not against concrete child implementations. The connector’s admissibility relies on monotonicity (Lemma 1) and the universal wiring-adequacy certificate defined in §3.4 (condition 5), together with the interaction-ownership and reduction-preservation disciplines detailed in §5.6. A connector that fails these conditions lies outside the calculus until an explicit coordinator or reduction proof is supplied.

4.3 Child refinement

Section titled “4.3 Child refinement”

A child refinement obligation along decomposition edge (v, c)(v, c) is

$$\alpha_{v,c}\left(\mathrm{beh}(M_c)\right) \subseteq \mathrm{beh}(B_{v,c}) \iff \mathrm{beh}(M_c) \subseteq \alpha^*_{v,c}\left(\mathrm{beh}(B_{v,c})\right).$$
αv,c(beh(Mc)) ⊆ beh(Bv,c) ⟺ beh(Mc) ⊆ αv,c∗(beh(Bv,c)).

When child M_cMc introduces local performance constraints or complexity bounds Φ_c^{cost}Φccost, the single relation handles it as a pullback-intersection inclusion:

$$\mathrm{beh}(M_c) \subseteq \alpha^*_{v,c}\left(\mathrm{beh}(B_{v,c})\right) \cap \Phi_c^{\mathrm{cost}}.$$
beh(Mc) ⊆ αv,c∗(beh(Bv,c)) ∩ Φccost.

When child M_cMc is a general-purpose component whose full input domain exceeds what parent vv‘s connector delivers, the edge obligation restricts to admissible input histories $A_{v,c}$Av,c:

$$\alpha_{v,c}\left(\mathrm{beh}(M_c) \restriction_{A_{v,c}}\right) \subseteq \mathrm{beh}(B_{v,c}),$$
αv,c(beh(Mc) ↾Av,c) ⊆ beh(Bv,c),

accompanied by input receptiveness and progress preservation on $A_{v,c}$Av,c.

For finite trace inclusion, an **inductive simulation witness** establishes it:

- an invariant $I_c \subseteq S_{M_c}$Ic ⊆ SMc;
- a relation $R_c \subseteq S_{M_c} \times S_{B_{v,c}}$Rc ⊆ SMc × SBv,c;
- an action abstraction map $\lambda_{v,c}$λv,c from child actions to contract actions or silent steps.

The conditions are:

1. **Initiation.** Every initial child state is in I_cIc and is related to an initial contract state.
2. **Consecution.** I_cIc is closed under the child's transitions.
3. **Step simulation.** For $(s,t) \in R_c$(s, t) ∈ Rc and $s \xrightarrow{\sigma} s'$s σ

   ---

   s′: if $\lambda_{v,c}(\sigma) = \epsilon$λv,c(σ) = ϵ, then $(s',t) \in R_c$(s′, t) ∈ Rc; otherwise there is t't′ with $t \xrightarrow{\lambda_{v,c}(\sigma)} t'$t λv,c(σ)

   ---

   t′ and $(s',t') \in R_c$(s′, t′) ∈ Rc.
4. **Input receptiveness.** Every contract input enabled at tt is enabled by some child input in $\lambda_{v,c}^{-1}$λv,c−1 at ss.
5. **Silent convergence.** For a liveness profile, a well-founded rank strictly decreases on silent steps, preventing infinite internal stuttering.

Lemma 3 (Finite-trace simulation). Conditions 1–3 imply $\lambda_{v,c}(\mathrm{tr}(M_c)) \subseteq \mathrm{tr}(B_{v,c})$λv,c(tr(Mc)) ⊆ tr(Bv,c).

Proof. Induct on the length of a child execution. Initiation relates the initial states. Consecution keeps the invariant. A silent step keeps the contract state; a visible step appends the matched contract transition. Every finite prefix therefore has a matching contract execution. ■ ■

For state-labeled properties, related states agree on the abstract valuations and silent steps preserve them. For liveness, the witness additionally preserves maximality and fairness, and validity condition 6 supplies the progress rank. For open inputs, simulation condition 4 blocks the trivial refusal strategy.

4.4 Realization

Section titled "4.4 Realization"

For each leaf cell $\ell \in V_{\mathrm{leaf}}$ℓ ∈ Vleaf, realization is certified by $\mathrm{beh}(P_\ell) \subseteq \mathrm{beh}(M_\ell)$beh(Pℓ) ⊆ beh(Mℓ) under the declared runtime semantics and environment assumptions. The substrate contract names what the runtime guarantees: an atomic write, a durable flush completion, or message delivery. For a decomposed parent cell, realization is discharged by the universal wiring-adequacy certificate (§3.4, condition 5), whose universal quantification over admissible child behaviors yields $\mathrm{Beh}_{\mathrm{exec}}(v) \subseteq K_v(\{\alpha_{v,c}(\mathrm{Beh}_{\mathrm{exec}}(c))\}_c)$Behexec(v) ⊆ Kv({αv,c(Behexec(c))}c) by application. For a leaf, it is the program-to-machine inclusion above.

4.5 Evidence discipline

Section titled "4.5 Evidence discipline"

An evidence record for an obligation oo is a tuple

$$e = (o,\ \mathrm{method},\ \mathrm{claim},\ \mathrm{result},\ \mathrm{scope},\ \mathrm{artifact}).$$
e = (o, method, claim, result, scope, artifact).

It is **valid** for oo when:

1. claimclaim is at least as strong as oo;
2. methodmethod is sound for claimclaim under scopescope;
3. resultresult is pass.

The result field is one of $\{\text{pass}, \text{fail}, \text{unresolved}, \text{pending}\}$. Only `pass` discharges an obligation; `unresolved` and `pending` are not passes and cannot enter the Verified outcome.

The method determines the claim:

Method	Claim it can establish	What it cannot establish
Proof derivation	the exact obligation	nothing beyond the logic's soundness
Exhaustive finite model check	the obligation for the modeled machine	adequacy of the model to the code
Bounded model check	the obligation up to the bound	the unbounded obligation
Property-based test	no violation in the sampled domain	a universal inclusion
Runtime monitor	no violation in the observed run	future or unobserved behavior
Translation validation	the specific program-to-machine relation	unrelated properties

The scope records the observation profile, machine class, environment assumptions, and any bound. A result is meaningful only within its scope.

4.6 Evidence separation and the three-gate structure

Section titled "4.6 Evidence separation and the three-gate structure"

Every development has three independent gates:

- **Provenance gate.** Every context premise $\Delta P_{\upsilon}\Delta\text{Pv}$ is valid, its upstream dependencies hold, and no load-bearing premise is stale.
- **Behavioral gate.** Every active intent has valid satisfaction evidence; every decomposition has valid evidence; every decomposition edge has valid child refinement evidence; every executable cell has valid realization evidence (wiring adequacy for a non-leaf cell, program-to-machine for a leaf); the profile's initialization, input, and (where claimed) progress conditions hold.
- **Preservation gate.** Every protected obligation in the previous version's effective inventory remains active with the same semantic content, or an authorized disposition records its retirement or weakening. A support, ownership, or interpretation edit that deactivates a protected obligation is rejected.

The separation matters. A failing behavioral check does not remove the intent from $A\text{A}$ (Lemma 2). A failing provenance check does not silently delete a machine. A support-only edit cannot deactivate a protected obligation without a disposition (condition 7). All failures are reported with their obligations, their evidence status, and the relevant witnesses. The version is not published until all three gates pass.

4.7 What the gate guarantees

Section titled "4.7 What the gate guarantees"

The gate guarantees that the published executable system refines the active root intents under the declared observation profile and substrate semantics. The guarantee is conditional on validity conditions 5 and 6 (executable realization and profile admissibility) and on the preservation condition 7, and it is relative to:

- the admitted root intents and their authority;
- the semantic profile and the environment assumptions;
- the soundness of the evidence checkers;
- the connector admissibility proofs;
- the declared execution semantics of the leaf programs.

These are the theorem's hypotheses, not disclaimers. The next section makes the composition conditions explicit.

---

5. Decomposition and Interaction

Section titled "5. Decomposition and Interaction"

5.1 Contracts are relational

Section titled “5.1 Contracts are relational”

A contract that constrains each child separately cannot express a correlation between children. The canonical witness is a parent that admits only the diagonal outcomes.

Proposition 1 (Coordinate projections erase correlation). Let the parent admit the outcome set $R = \{(0,0),(1,1)\}$R = {(0, 0), (1, 1)} over outputs (x, y)(x, y). The coordinate projections are $\pi_1(R) = \{0,1\}$π1(R) = {0, 1} and $\pi_2(R) = \{0,1\}$π2(R) = {0, 1}. Two children that independently refine the projections can jointly emit $(0,1) \notin R$(0, 1) ∈/ R.

Proof. $\{0,1\} \times \{0,1\}${0, 1} × {0, 1} contains $(0,1)$(0, 1), which is not in RR. ■■

The remedy is one of two moves: make the contract relational (a relation between child observations and shared connector state), or hoist the correlation into the connector’s wiring. The calculus supports both. The point is that the parent cell must own the correlation; it cannot be pushed into independent child proofs.

5.2 Sibling independence

Section titled “5.2 Sibling independence”

Theorem 2 (Sibling independence). Fix a parent cell $v \in V$v ∈ V with children $\mathrm{ch}(v)$ch(v), contracts $(B_{v,c})_{c \in \mathrm{ch}(v)}$(Bv,c)c∈ch(v), wiring W_v Wv, and parent machine M_vMv. Suppose:

1. **Child refinement.** For every $c \in \mathrm{ch}(v)$c ∈ ch(v), $\alpha_{v,c}(\mathrm{beh}(M_c)) \subseteq \mathrm{beh}(B_{v,c})$αv,c(beh(Mc)) ⊆ beh(Bv,c), with input receptiveness and progress preservation under the profile.
2. **Connector monotonicity.** The transformer K_vKv induced by W_vWv is monotone.
3. **Decomposition.** $K_v(\{\mathrm{beh}(B_{v,c})\}_c) \subseteq \mathrm{beh}(M_v)$Kv({beh(Bv,c)}c) ⊆ beh(Mv).
4. **Applicability.** The wiring’s operational realization is sound for K_vKv; interaction ownership and reduction preservation hold as required by the profile.

Then

$$K_v\left(\{\alpha_{v,c}(\mathrm{beh}(M_c))\}_c\right) \subseteq \mathrm{beh}(M_v),$$
Kv({αv,c(beh(Mc))}c) ⊆ beh(Mv),

without constructing the concrete sibling product $\prod_c S_{M_c}$∏c SMc.

Proof. Apply the abstraction maps to the child refinements, then apply monotonicity to get $K_v(\{\alpha_{v,c}(\mathrm{beh}(M_c))\}_c) \subseteq K_v(\{\mathrm{beh}(B_{v,c})\}_c)$Kv({αv,c(beh(Mc))}c) ⊆ Kv({beh(Bv,c)}c), and compose with the decomposition. ■■

The theorem is the step case of Theorem 1, stated separately because it is the engineering claim. The children are verified against their contracts, never against each other. The cost of the parent check is the abstract boundary, not the concrete product.

5.3 Executable closure

Section titled “5.3 Executable closure”

A connector that performs stateful coordination cannot remain an unimplemented mathematical object. The calculus requires:

- the wiring W_vWv owns only the specified composition mechanism: routing, synchronizing, interleaving, hiding;
- any stateful coordinator is a child cell with its own machine, contract, and eventual realization;
- any custom router, scheduler, or runtime adapter introduced by the implementation receives its own realization obligation or becomes an explicit substrate assumption;
- no action disappears between the mathematical composition and the executable delivery.

This rule closes the gap that otherwise lets an agent write a beautiful abstract connector and never implement it. The coordinator is refined like everything else.

5.4 Shared services

Section titled “5.4 Shared services”

A service used by several branches in the DAG is declared at a selected common dominating ancestor uu (an ancestor $u \in \bigcap_{p \in \mathrm{par}(c)} \mathrm{Anc}(\{p\})$u ∈ ∩p∈par(c) Anc({p}) through which all paths from V_{root}Vroot to cc pass) as an abstract atomic contract O^{abs}Oabs in the ambient context. Clients invoke it atomically without widening intermediate interfaces. The service is implemented once, as a separate subgraph, and proved to refine O^{abs}Oabs: linearizability for safety, a progress property for liveness. This is a factoring rule, not a new judgment. A stateless both–mover service (logging, telemetry) is an admissible connector with trivial state; its certificates establish the required commutation and progress.

5.5 Contract factoring and capacity

Section titled “5.5 Contract factoring and capacity”

A contract boundary can move history into the child and keep the connector small. With mm idempotent resource bits and the invariant “not all mm are held,” a naive joint recognizer needs 2^m2m states. With per–child two–state contracts that emit signed changes only on a genuine bit change, a connector that counts holders needs $m + 1$m + 1 states. The gain is architectural: the connector’s state is small, and the history lives in the children. The bound is not a universal polynomial claim; the abstract boundary can still contain a product of contract states when the architecture requires it. The point is to choose the boundary so that the product is small.

5.6 Applicability conditions

Section titled “5.6 Applicability conditions”

The sibling–independence theorem is only as strong as its applicability conditions. The paper states them in the main text because they are the load–bearing assumptions:

- **Interaction ownership.** Every shared interaction has an owning boundary; children have disjoint private state.
- **Operational soundness.** The wiring model covers actual routing, synchronization, initialization, and shared–resource behavior.
- **Input and progress.** The profile’s input receptiveness and progress obligations are preserved by the child refinement.
- **Reduction preservation.** When interleavings are reduced, the reduction preserves the observations of the profile.

A connector that fails one of these is outside the calculus. The failure is a proof obligation, not a silent assumption.

---

6. The Development Protocol

Section titled “6. The Development Protocol”

6.1 The five gates

Section titled “6.1 The five gates”

The protocol is the operational reading of validity. An agent may publish a version only when all five gates pass.

G1 — Read the context (process duty). Before proposing a change at a cell vv, the agent reads the cell’s context premises ΔP_vΔPv, its active intents $A(v)$A(v), support links JJ, incoming contracts, outgoing decomposition $\chi(v)$χ(v), and the cut/subgraph view in which vv participates. This is a process obligation on the agent; G2—G5 are checked by the gate.

G2 — No abstraction skip. A machine at child cell cc is introduced only as a child of a contract already declared in some decomposition record $\chi(v)$χ(v). The agent cannot write leaf code without first fixing the contract it must satisfy. This is the direct block on the wide abstraction leap.

G3 — Evidence required. Acceptance of a cell requires valid satisfaction evidence for every active intent assigned to it and valid executable–realization evidence: for a non–leaf cell, the universal wiring–adequacy certificate of condition 5; for a leaf, the program–to–machine inclusion. A non–leaf cell also requires valid decomposition evidence. Acceptance of a decomposition edge requires valid child refinement evidence. A test is evidence only for the claim it can establish. A weakened test weakens the claim, not the obligation; the universal obligation remains active and unsatisfied.

G4 — No silent specification weakening. Root and protected intents and contracts can be changed only through an authorized revision that creates a new version and reopens the dependent context premises. The agent may propose a revision; it cannot apply one as part of a code change. A support, ownership, or interpretation edit that deactivates a protected obligation counts as a specification weakening and requires an authorized disposition. The authorization record is part of the version.

G5 — Atomic publication. A version is published only when every obligation in the affected dependency cone has valid pass evidence. Incomplete subgraphs are not publishable. Missing, unresolved, or pending evidence is not a pass, and a failed local verification cannot be hidden by a successful build elsewhere.

6.2 The authoring cycle

Section titled “6.2 The authoring cycle”

The agent’s authoring cycle is:

1. **Read.** Load the cell’s cut inspection view and incoming slot contracts.
2. **Context.** For each cell, record context premises ΔP_vΔPv with upstream dependencies, alternatives, rationale, and reopening conditions. An edit that changes a protected item requires an authorized revision.
3. **Model.** Propose the cell’s machine M_vMv. The machine must satisfy the active intents assigned to the cell.
4. **Decompose.** For an internal cell, declare the wiring skeleton and child contracts, and discharge the decomposition and wiring–adequacy obligations.
5. **Assign.** Hand each child contract to the child cell. The child may be developed independently within its contract.
6. **Substitute.** When a child is complete, substitute its refinement proof into the parent’s decomposition. Close the parent only when all child edges are discharged.

A locally checked design with unresolved descendants remains a **development candidate**. The completed–system gate quantifies over all required cells, decompositions, and executable artifacts.

6.3 What the agent may and may not change

Section titled “6.3 What the agent may and may not change”

Artifact	Agent may	Agent may not
Delegated intent	refine, strengthen, or add a child intent	delete or deactivate it via a support/ownership edit without an authorized revision
Protected intent	propose a revision with rationale and reopening	apply the revision, or deactivate it via a support/ownership edit, without an authorized disposition
Root intent	implement it	silently weaken, reinterpret, or deactivate it
Contract	refine the child implementation; propose a contract revision	change the contract to make a failing child pass
Context premise	add, re–warrant, or reopen with rationale	erase the alternatives or the rationale
Machine	choose any implementation within the contract	declare a contract that the parent decomposition does not support
Evidence	add evidence for the exact obligation	substitute a weaker claim for a stronger obligation

6.4 Test weakening, intent revision, and authority

Section titled “6.4 Test weakening, intent revision, and authority”

The test–weakening failure is blocked at the evidence layer. A test is a decision procedure for a bounded or sampled claim. It cannot discharge a universal inclusion unless it is exhaustive for the relevant finite domain. If the agent deletes an assertion, the remaining test establishes a weaker claim; the universal obligation is still active, and the gate reports it as unsatisfied. The only way to remove the obligation is to change its intent, which requires an authorized revision and reopens the dependent context premises. A support, ownership, or interpretation edit that removes a protected obligation is a removal even when the predicate bytes are unchanged; it requires the same authorized disposition.

The authority tags make this concrete:

- **Root intents** are admitted by the owner at version creation.
- **Protected intents and contracts** require an explicit authorization record to change.
- **Delegated intents** may be refined by the agent within the protected boundary.

The authorization record is not a separate governance layer. It is a field of the version, checked by the same gate.

6.5 A transcript of a caught error

Section titled “6.5 A transcript of a caught error”

1. The requirement says: acknowledge a request only after its result is durable.
2. The agent writes a store that emits `committed` after `write` returns.
3. The refinement check for the store contract returns the trace `append, committed` with no `flush_ok`/`fsync`.
4. The evidence record is fail. The store contract remains active.
5. The agent either adds the flush before `committed` (downward repair at the code layer) or, if the substrate cannot support the required durability, introduces a supported flush protocol and implements it, or obtains an authorized revision of the substrate/requirement premise and reopens the store decomposition.
6. The version is published only after the new evidence passes and the reopen path is recorded.

The next section gives the full example.

---

7. Versioned Evolution and Top–Down Projection

Section titled “7. Versioned Evolution and Top–Down Projection”

7.1 Versioned Refinement DAGs

Section titled “7.1 Versioned Refinement DAGs”

Let U_{cell}Ucell be a shared universe of immutable cell interfaces, state machines, and wiring specifications. A versioned system is mathematically defined as a version–indexed sequence of Refinement DAG developments

$$\{G_t\}_{t \in N}$$
{Gt}t∈N

over this universe. Each version

$$G_t = \left(V^{(t)},\ M_t,\ \chi_t,\ I^{(t)},\ J^{(t)},\ E_t,\ A_t\right)$$
Gt = (V (t), Mt, χt, I(t), J(t), Et, At)

is an immutable Refinement DAG at evolution epoch tt, accompanied by its authorization record A_tAt for protected revisions. Each active cell $v \in V^{(t)}$v ∈ V (t) carries:

1. a **local model** $M_t(v) = \left(O_{X_{t,v}}, \Phi^{(t)}(v), M_v^{(t)}, \Gamma_v^{(t)}\right)$Mt(v) = (OXt,v , Φ(t)(v), Mv(t), Γv(t)), comprising its observation space at versioned interface $X_{t,v}$Xt,v, active intent conjunction, state machine, and semantic context $\Gamma_v^{(t)}$Γv(t);

2. a **typed decomposition record** $\chi_t(v) = \left(W_{t,v}, \mathrm{ch}_t(v), (B_{t,v,c}, \alpha_{t,v,c})_{c \in \mathrm{ch}_t(v)}\right)$χt(v) = (Wt,v, cht(v), (Bt,v,c, αt,v,c)c∈cht(v)) for non-leaf vv, specifying its wiring skeleton $W_{t,v}$Wt,v, children $\mathrm{ch}_t(v) \subseteq V^{(t)}$cht(v) ⊆ V (t), child contracts $B_{t,v,c}$Bt,v,c at slot interfaces $X_{t,v,c}$ Xt,v,c, child context profiles $\Gamma_{t,c}$Γt,c, and child edge abstraction maps $\alpha_{t,v,c}$αt,v,c. At epoch tt, concrete wiring implementation $C_{t,v}$ Ct,v realizes connector transformer $K_{t,v} = K_{W_{t,v}}$Kt,v = KWt,v.

Structural evolution from version tt to version $t+1$t + 1 is modeled purely as an update of local models M_{t+1}Mt+1 and decompositions χ_{t+1}χt+1 on an affected subgraph, preserving unchanged components by mathematical identity. All canonical structural transformations are represented uniformly:

- $1 \to 1$**1 → 1 modification.** A cell vv modifies its internal machine $M_v^{(t+1)}$Mv(t+1) or modifies an incoming contract $B_{v,c}$Bv,c, while the child set $\mathrm{ch}_{t+1}(v) = \mathrm{ch}_t(v)$cht+1(v) = cht(v) remains isomorphic.
- $1 \to M$**1 → M splitting.** A single cell $c \in \mathrm{ch}_t(v)$c ∈ cht(v) is partitioned into MM finer-grained modular children $\{c_1, \ldots, c_M\} \subset V^{(t+1)}$ {c1, … , cM} ⊂ V (t+1), updating parent wiring $W_{t+1,v}$Wt+1,v and assigning individual child contracts B_{t+1,v,c_j}Bt+1,v,cj.
- $N \to 1$**N → 1 merging.** Multiple sibling components $\{c_1, \ldots, c_N\} \subset \mathrm{ch}_t(v)$ {c1, … , cN} ⊂ cht(v) are consolidated into a single unified child cell $c^* \in V^{(t+1)}$c∗ ∈ V (t+1), collapsing sibling contracts into a single interface.
- **Cross-boundary** $N \to M$**N → M refactoring.** Restructuring component boundaries across multiple cells is represented directly by updating χ_{t+1}χt+1 on the affected sub-DAG.

Because versions are complete mathematical developments over an immutable component universe, these transformations require **zero** `next` forwarding pointers between obsolete and new objects, **zero** intermediate transition nodes τ_tτt cluttering the refinement graph, and **zero** birth-death interval encodings (alive$(v) = [t_{\mathrm{birth}}, t_{\mathrm{death}}]$alive(v) = [tbirth, tdeath]).

Abandonment and rollback. An unsuccessful or discarded candidate version tt is abandoned by simply defining $G_{t+1} = G_s$Gt+1 = Gs on the reverted components, where $s < t$s < t is the chosen target historical checkpoint. Every historical version—whether accepted, superseded, or abandoned—lives uniformly in the sequence $\{G_t\}_{t \in N}$ {Gt}t∈N. No branch-status filters, active/dead flags, or tombstones are needed to distinguish historical states: the active system at epoch tt is identically G_tGt.

7.2 Contextual intent projection and property-driven refinement

Section titled “7.2 Contextual intent projection and property-driven refinement”

When a requirement evolves or a new capability is introduced, intent propagation proceeds strictly along the DAG from roots through successive child edges, without invoking any global heuristic classifier. At each step, an agent refines an incoming component requirement by decomposing it into sibling contracts under local domain premises.

Definition (Contextual Intent Projection). Let cell $v \in V^{(t)}$v ∈ V (t) have an active component requirement $R_v \subseteq O_{X_v}$Rv ⊆ OXv and local context premises ΔP_vΔPv (encapsulating domain axioms, maintained state invariants, or declared workload conditions). A decomposition $\chi_{t+1}(v) = \left(W_{t+1,v}, \mathrm{ch}_{t+1}(v), (B_{t+1,v,c}, \alpha_{t+1,v,c})_{c \in \mathrm{ch}_{t+1}(v)}\right)$χt+1(v) = (Wt+1,v, cht+1(v), (Bt+1,v,c, αt+1,v,c)c∈cht+1(v)) is a **valid contextual projection** of R_vRv under ΔP_vΔPv iff

$$\Delta P_v \implies K_{W_{t+1,v}}\left(\left\{\mathrm{beh}(B_{t+1,v,c})\right\}_{c \in \mathrm{ch}_{t+1}(v)}\right) \subseteq R_v.$$

ΔPv ⟹ KWt+1,v({beh(Bt+1,v,c)}c∈cht+1(v)) ⊆ Rv.

Residual Conservation Law. The child contracts $\{B_{t+1,v,c}\}_{c \in \mathrm{ch}_{t+1}(v)}$ {Bt+1,v,c}c∈cht+1(v) *are* the residual obligations passed down to children. When cell vv extracts no intermediate abstraction, the projection is the identity pass-through $B_{t+1,v,c} = R_v$Bt+1,v,c = Rv with a single child. The inclusion guarantees conservation of intent across abstraction boundaries: an agent cannot satisfy an abstract goal by relaxing a binding mechanism requirement without explicitly discharging the residual constraint in the lower layer.

Maintained behavioral invariants are enforced as subset inclusions across sibling contracts and connector wiring, whereas environmental and workload premises are recorded in ΔP_vΔPv. If operational workload or environment assumptions shift, premise ΔP_vΔPv becomes

stale, triggering targeted reopening of the local decomposition without invalidating orthogonal verified components.

7.3 Exact obligation propositions and the incremental frame theorem

Section titled “7.3 Exact obligation propositions and the incremental frame theorem”

Let G_tGt be a Refinement DAG. The mathematical obligations required by validity (§3.4–3.5) are defined as a finite set of exact, scoped behavioral–inclusion propositions:

$$\mathrm{Req}(G_t) = \mathrm{Sat}(G_t) \cup \mathrm{Wire}(G_t) \cup \mathrm{Dec}(G_t) \cup \mathrm{Ref}(G_t) \cup \mathrm{Impl}(G_t),$$
Req(Gt) = Sat(Gt) ∪ Wire(Gt) ∪ Dec(Gt) ∪ Ref(Gt) ∪ Impl(Gt),

where each constituent proposition is fully determined by its semantic operands:

- $\mathrm{Sat}(v, c)\colon = \left(\mathrm{beh}(M_v^{(t)}) \subseteq \left[\!\left[\, c \,\right]\!\right] \text{ under } \Gamma_v^{(t)}\right)$Sat(v, c) := (beh(Mv(t)) ⊆ ⟦c⟧ under Γv(t)) for $v \in V^{(t)}, c \in A^{(t)}(v)$v ∈ V (t), c ∈ A(t)(v);
- $\mathrm{Wire}(v)\colon = \left(\forall (Q_c)_{c \,\in\, \mathrm{ch}_t(v)}.\ \left(\forall c \in \mathrm{ch}_t(v).\ Q_c \text{ is admissible for } X_{t,v,c}, \Gamma_{t,c}\right) \implies \mathrm{Beh}(C_{t,v}[Q_c]) \subseteq K_{t,v}\left(\{\alpha_{t,v,c}(Q_c)\}_c\right)\right)$Wire(v) := (∀(Qc)c∈cht(v). (∀c ∈ cht(v). Qc is admissible for Xt,v,c, Γt,c) ⟹ Beh(Ct,v[Qc]) ⊆ Kt,v({αt,v,c(Qc)}c)) for non-leaf $v \in V^{(t)}$ v ∈ V (t);
- $\mathrm{Dec}(v)\colon = \left(K_{t,v}(\{\mathrm{beh}(B_{t,v,c})\}_{c \,\in\, \mathrm{ch}_t(v)}) \subseteq \mathrm{beh}(M_v^{(t)}) \text{ under } \Gamma_v^{(t)}\right)$Dec(v) := (Kt,v({beh(Bt,v,c)}c∈cht(v)) ⊆ beh(Mv(t)) under Γv(t)) for non-leaf $v \in V^{(t)}$v ∈ V (t);
- $\mathrm{Ref}(v, c)\colon = \left(\alpha_{t,v,c}(\mathrm{beh}(M_c^{(t)})) \subseteq \mathrm{beh}(B_{t,v,c}) \text{ under } \Gamma_{t,c}\right)$Ref(v, c) := (αt,v,c(beh(Mc(t))) ⊆ beh(Bt,v,c) under Γt,c) for $v \in V^{(t)}, c \in \mathrm{ch}_t(v)$v ∈ V (t), c ∈ cht(v);
- $\mathrm{Impl}(\ell)\colon = \left(\mathrm{beh}(P_\ell^{(t)}) \subseteq \mathrm{beh}(M_\ell^{(t)}) \text{ under } \Gamma_\ell^{(t)}\right)$Impl(ℓ) := (beh(Pℓ(t)) ⊆ beh(Mℓ(t)) under Γℓ(t)) for leaf $\ell \in V_{\mathrm{leaf}}^{(t)}$ℓ ∈ Vleaf(t).

For a development version G_tGt, a proposition $q \in \mathrm{Req}(G_t)$q ∈ Req(Gt) holds in version tt, written $\mathrm{Holds}_t(q)$Holdst(q), when its behavioral inclusion is mathematically satisfied and witnessed by a valid evidence record in E_tEt (§4.5).

Theorem 3 (Incremental Frame and Verification Cutoff). Let G_tGt be a valid Refinement DAG, and let G_{t+1}Gt+1 be a candidate Refinement DAG satisfying conditions 1–2 and 6–7 of §3.4 (support/context, profile admissibility, and protected preservation), where evidence for unchanged obligations $q \in \mathrm{Req}(G_{t+1}) \cap \mathrm{Req}(G_t)$q ∈ Req(Gt+1) ∩ Req(Gt) is carried over by identity into E_{t+1}Et+1 ($E_{t+1}(q) = E_t(q)$Et+1(q) = Et(q)) with preserved applicability. If every newly introduced or modified obligation in the set difference holds in G_{t+1}Gt+1:

$$\forall q \in \mathrm{Req}(G_{t+1}) \setminus \mathrm{Req}(G_t), \quad \mathrm{Holds}_{t+1}(q),$$
∀q ∈ Req(Gt+1) ∖ Req(Gt), Holdst+1(q),

then every obligation in $\mathrm{Req}(G_{t+1})$Req(Gt+1) holds in G_{t+1}Gt+1 ($\forall q \in \mathrm{Req}(G_{t+1}).\ \mathrm{Holds}_{t+1}(q)$∀q ∈ Req(Gt+1). Holdst+1(q)), G_{t+1}Gt+1 is a valid Refinement DAG, and by Theorem 1 its executable root behavior refines all active root intents: for all $r \in V_{\mathrm{root}}^{(t+1)}$r ∈ Vroot(t+1), $\mathrm{Beh}_{\mathrm{exec}}(r) \subseteq \Phi(r)$Behexec(r) ⊆ Φ(r).

Proof. Partition $\mathrm{Req}(G_{t+1})$Req(Gt+1) into two disjoint subsets:

$$\mathrm{Req}(G_{t+1}) = \left(\mathrm{Req}(G_{t+1}) \cap \mathrm{Req}(G_t)\right) \ \cup \ \left(\mathrm{Req}(G_{t+1}) \setminus \mathrm{Req}(G_t)\right).$$
Req(Gt+1) = (Req(Gt+1) ∩ Req(Gt)) ∪ (Req(Gt+1) ∖ Req(Gt)).

For any $q \in \mathrm{Req}(G_{t+1}) \cap \mathrm{Req}(G_t)$q ∈ Req(Gt+1) ∩ Req(Gt), qq was an obligation of G_tGt. Since G_tGt is valid, $\mathrm{Holds}_t(q)$Holdst(q) was witnessed by valid evidence $E_t(q)$Et(q) with applicable scope. Because qq is identical in semantic operands, $E_t(q)$Et(q) carried over to $E_{t+1}(q)$Et+1(q) remains valid and applicable, giving $\mathrm{Holds}_{t+1}(q)$Holdst+1(q). For any $q \in \mathrm{Req}(G_{t+1}) \setminus \mathrm{Req}(G_t)$q ∈ Req(Gt+1) ∖ Req(Gt), the hypothesis directly asserts $\mathrm{Holds}_{t+1}(q)$Holdst+1(q). Thus every obligation $q \in \mathrm{Req}(G_{t+1})$q ∈ Req(Gt+1) holds in G_{t+1}Gt+1.

Combined with conditions 1–2 and 6–7, G_{t+1}Gt+1 satisfies all validity conditions 1–7 of §3.4 and edge refinement of §3.5. By Theorem 1 (Refinement–DAG Soundness), $\mathrm{Beh}_{\mathrm{exec}}(r) \subseteq \Phi(r)$Behexec(r) ⊆ Φ(r) for all roots $r \in V_{\mathrm{root}}^{(t+1)}$r ∈ Vroot(t+1). ■■

Corollary 2 (Interface Cutoff). Let G_tGt be a valid development. Suppose a sub–development rooted at child $c \in \mathrm{ch}(v)$c ∈ ch(v) is replaced by an updated sub–development ($1 \to 11 \to 1$, $1 \to M1 \to M$, $N \to 1N \to 1$, or $N \to MN \to M$) while preserving the incoming contract $B_{v,c}$Bv,c, parent wiring W_vWv, parent machine M_vMv, abstraction map $\alpha_{v,c}$αv,c, contexts, and sibling contracts. Then:

1. $\mathrm{Dec}(v)$Dec(v), $\mathrm{Wire}(v)$Wire(v), $\mathrm{Sat}(v, \cdot)$Sat(v, ·), and all ancestor obligations belong to $\mathrm{Req}(G_{t+1}) \cap \mathrm{Req}(G_t)$Req(Gt+1) ∩ Req(Gt) and require **zero ancestor solver re–execution**;
2. any newly introduced child machines M'_{c_j}Mcj′ or leaf implementations P'_ℓPℓ′ enter $\mathrm{Req}(G_{t+1}) \setminus \mathrm{Req}(G_t)$Req(Gt+1) ∖ Req(Gt) and must be discharged.

7.4 Implementation remark: persistent outgoing–edge containers

Section titled “7.4 Implementation remark: persistent outgoing–edge containers”

In an executable repository or development environment, version–indexed decompositions $\chi_t(v)$χt(v) are realized without copying entire tree spines by maintaining a **persistent array or persistent linked list of outgoing decomposition entries** at each cell, with a fixed sentinel root $r_\top$r⊤ above V_{root}Vroot:

- **Localized update.** Updating a region at version $t+1$t + 1 appends a new versioned entry $(t+1, \chi_{t+1}(v))$(t + 1, χt+1(v)) to the upper–boundary parent’s persistent container. Unchanged ancestors and sibling subtrees remain shared by pointer identity without cloning.
- **Rollback.** Reverting a sub–development to version $s < t$s < t appends $(t+1, \chi_s(v))$(t + 1, χs(v)) as an $O(1)$O(1) reference copy.
- **Historical inspection.** Reading the development as of any historical or current epoch TT unconditionally selects the decomposition entry with the largest index $t \le T$t ⩽ T at each visited cell.

7.5 The evolution loop

Section titled “7.5 The evolution loop”

The evolution function

$$G_{t+1} = \mathrm{evolve}(G_t, \Delta I)$$
Gt+1 = evolve(Gt, ΔI)

computes a candidate development from an accepted delta ΔIΔI, returning **Verified**, **Conflict**, or **Unresolved**. There is no default–to–pass.

Step 1 — Apply and reactivate. Apply ΔIΔI: a new root intent, an updated context premise, an authorized revision, or a contract adjustment. Recompute active intents AA via topological support evaluation. Evaluate affected context premises ΔP_vΔPv against upstream models and verify mutual compatibility. Mark stale components by reachability along ΔP_v^*ΔPv∗. If context premises conflict, return **Conflict** with the conflicting premises and rationales; stop before initiating machine search.

Step 2 — Locate. Walk the DAG in topological order, evaluating only the newly introduced or modified obligations in the set difference $\mathrm{Req}(G_{t+1}) \setminus \mathrm{Req}(G_t)$Req(Gt+1) ∖ Req(Gt). The result of each check is one of: **pass**, **fail** with a semantic counterexample, **failed proof witness** (e.g., an inductive invariant or simulation relation that failed), or **unresolved** (timeout or resource exhaustion). Let jj be the first cell at which an obligation does not pass. Publish only when every required obligation in $\mathrm{Req}(G_{t+1}) \setminus \mathrm{Req}(G_t)$Req(Gt+1) ∖ Req(Gt) has valid pass evidence.

Step 3 — Repair at jj. The check result determines the repair strategy:

- **Semantic counterexample, genuine.** A concrete execution admitted by cell jj‘s machine violates an active intent, or a concrete interleaving of child contracts violates the decomposition. Repair the machine, connector, or contracts at cell jj; re–establish its obligations and upward refinement; propagate upward along the affected spine.

- **Semantic counterexample, spurious child refinement.** The child's model or abstraction overapproximates: it admits behavior the fixed child implementation cannot produce. Repair the child model or its abstraction map by narrowing the abstraction, and re-establish the lower realization and upper refinement. Narrowing the parent-side contract for a fixed child abstraction makes the inclusion harder and cannot remove the spurious behavior.
- **Semantic counterexample, spurious decomposition.** The parent-side child contract is too permissive and the concrete children satisfy a stronger contract. Narrow the parent-side child contract or repair the connector, and re-establish the decomposition. If the concrete children do not satisfy the stronger contract, the repair belongs to the children.
- **Failed proof witness.** An inductive invariant or simulation relation failed. Strengthen or repair the witness and recheck. This is a proof-search outcome, not a semantic counterexample and not a machine edit by itself.
- **Unresolved.** The checker timed out or returned unknown. Return **Unresolved**; do not classify as genuine or spurious, and do not publish.

For a refinement failure, the discrete set example is decisive: implementation $\{ok\}$, child abstraction $\{ok, bad\}$, parent contract $\{ok\}$. The implementation satisfies the contract, but the abstraction admits bad. No stronger parent contract repairs the fixed child abstraction; narrowing the child abstraction to $\{ok\}$ does. A child refinement failure is never repaired by strengthening the parent-side contract alone.

The genuine/spurious test for a semantic decomposition counterexample uses one shared trajectory:

$$\exists r.\ \mathrm{legal}_{W_v}(r) \ \wedge\ \mathrm{obs}_v(r) = \tau \ \wedge\ \forall c \in \mathrm{ch}(v).\ \mathrm{slice}_c(r) \in \mathrm{beh}(M_c),$$

writing $\mathrm{realizable}(M_c, o) \iff o \in \mathrm{beh}(M_c)$ to indicate that observation slice o is realizable by child machine M_c. A separate existential trajectory per child is not sufficient: two children can each be realizable on their own slice while no common trajectory exists. The shared-bit witness is Proposition 2.

Step 4 — Publish. If every obligation in $\mathrm{Req}(G_{t+1}) \setminus \mathrm{Req}(G_t)$ has valid pass evidence and every load-bearing context premise is warranted, atomically install G_{t+1} and record authorization A_{t+1}. Otherwise return **Unresolved** with the failing or unresolved obligations and their witnesses.

7.6 Genuine vs. spurious

Section titled "7.6 Genuine vs. spurious"

Proposition 2 (Separate realizations do not imply a joint realization). There are two child contracts, one requiring a shared bit to be 0 and one requiring it to be 1, such that each is separately realizable but no common trajectory realizes both.

Proof. Let the shared bit be $b \in \{0, 1\}$. Child 1 is realizable only with $b = 0$; child 2 is realizable only with $b = 1$. Each existential check succeeds with a different value of b. No single r has $b = 0$ and $b = 1$. ■■

The proposition is the reason the repair step uses a joint trajectory. It is also the reason a connector that hides shared state behind per-child contracts is unsound: the correlation is not in either contract.

7.7 Staleness and reopening

Section titled "7.7 Staleness and reopening"

A change to an intent, contract, or context premise activates its dependency cone along ΔP_v^*. Every cell in the cone is stale until re-warranted. A reopened design alternative becomes selectable again under updated context premises; selecting it creates an updated decomposition with its own context and evidence. Prior versions remain in the history $\{G_t\}$, preserving the audit trail.

The dependency cone is computed on the reference graph of semantic operands, not on physical storage pointers. An internal implementation change with an unchanged semantic contract does not activate the cone. This is the practical benefit of grounding obligations directly in semantic proposition identity: unchanged boundaries immediately prune propagation, keeping the verified blast radius strictly localized to the modified subgraph.

8. End–to–End Example: Durable Acknowledgement

Section titled “8. End–to–End Example: Durable Acknowledgement”

8.1 The requirement and scope

Section titled “8.1 The requirement and scope”

The system instance serves one request. The environment precondition is: at most one `req(id)` is delivered to an instance, and the instance is not reused after `ack` or `fail`. A reusable service needs a reset transition and matching contracts; that variant is outside this example.

The declared safety intent is:

- **Safety.** For every identifier idid, an `ack(id)` output occurs only after the result for idid is durable, witnessed at the store boundary by `flush_ok(id)` before `committed(id)`.

The liveness claim is separate and conditional:

- **Conditional liveness.** If the scheduler is fair, the store responds to every `append(id)` with `flush_ok(id)` or `flush_fail(id)`, and the coordinator is fair, then every `req(id)` is eventually followed by `ack(id)` or `fail(id)`.

The safety proof does not establish conditional liveness. Liveness is discharged only under those premises, which are recorded in the profile of the coordinator and store cells. The leaf runtime is run–to–completion, non–reentrant, with a FIFO event queue: `emit` enqueues an event and returns; the current handler completes before the next event is dispatched. Initialization is coordinator state `idle` and store state `empty`. The reentrant interpretation is outside this example; §8.5 also places the coordinator in `appending` before `append` is emitted, so a reentrant response is not discarded.

The architecture is modeled as a 2–level Refinement DAG without dummy padding: root cell v_0v0 decomposes into children $\mathrm{ch}(v_0) = \{\mathrm{coord}, \mathrm{store}\}$ch(v0) = {coord, store}, both of which are leaf cells ($\mathrm{ch}(\mathrm{coord}) = \mathrm{ch}(\mathrm{store}) = \varnothing$ch(coord) = ch(store) = ∅) realized directly by executable programs P_{coord}Pcoord and P_{store}Pstore.

8.2 Root cell: the abstract design

Section titled “8.2 Root cell: the abstract design”

The root cell v_0v0 has the machine M_0M0:

- states: `idle`, `pending(id)`, `durable(id)`, `acked(id)`;
- transitions:
  - `idle --req(id)--> pending(id)`;
  - `pending(id) --commit_ok(id)--> durable(id)`;
  - `pending(id) --commit_fail(id)--> acked(id)` with output `fail(id)`;
  - `durable(id) --tau--> acked(id)` with output `ack(id)`.

The root has no transition out of `acked(id)`; the instance is single–use, matching the environment precondition. The machine satisfies the safety intent by construction: `ack(id)` is emitted only from `durable(id)`, which is reached only by `commit_ok(id)`. Conditional liveness is a separate obligation under the fairness premises; it is not inferred from this safety construction.

The root context premises ΔP_{v_0}ΔPv0 record why the design separates durability from acknowledgement: the alternative of acknowledging on enqueue was rejected because it treats a volatile queue as durable; the reopening condition is a substrate that provides a durable atomic enqueue.

8.3 Root decomposition

Section titled “8.3 Root decomposition”

The root cell v_0v0 declares two children: a coordinator `coord` and a durable store `store`. The wiring W_0W0 routes:

- `req(id)` from the environment to `coord`;
- `append(id)` from `coord` to `store`;
- `committed(id)` and `failed(id)` from `store` to `coord`;
- `ack(id)` and `fail(id)` from `coord` to the environment.

The store's boundary also exposes `flush_ok(id)` and `flush_fail(id)`; the wiring routes them to the environment observer that witnesses durability. The root's `commit_ok` is the abstraction of the store's `flush_ok` followed by `committed`; the abstraction map αα is part of the decomposition record.

The contracts are:

$B_{0,\,\mathrm{coord}}$**B0,coord** — inputs `req(id)`, `committed(id)`, `failed(id)`; outputs `append(id)`, `ack(id)`, `fail(id)`. Duties: at most one `req(id)` per instance; no second request after `ack` or `fail`; `append(id)` may follow `req(id)`; `ack(id)` only after `committed(id)`; `fail(id)` only after `failed(id)`. Conditional progress: under fair coordinator scheduling and a store response, every `req(id)` eventually receives `ack(id)` or `fail(id)`.

$B_{0,\,\mathrm{store}}$**B0,store** — inputs `append(id)`; observable outputs `flush_ok(id)`, `flush_fail(id)`, `committed(id)`, `failed(id)`. Duties: at most one `append(id)` per instance; `committed(id)` only after `flush_ok(id)`; `flush_ok(id)` is the substrate's durability event; `failed(id)` only after `flush_fail(id)`. Conditional progress: under the substrate's fairness, every `append(id)` eventually receives `flush_ok(id)` or `flush_fail(id)` and then `committed(id)` or `failed(id)`.

The contracts are not merely necessary ordering duties. Their trace languages are the prefix closures (where $\mathrm{pref}(L) = \{u \mid \exists w.\ uw \in L\}$ pref(L) = {u | ∃w. uw ∈ L} denotes the prefix closure of trace set LL), with one identifier per instance, alongside the root machine's trace language L_0L0:

$$
\begin{aligned}
L_{\mathrm{coord}} &= \mathrm{pref}(\mathrm{req}(id)\,\mathrm{append}(id)\,\mathrm{committed}(id)\,\mathrm{ack}(id)) \cup \mathrm{pref}(\mathrm{req}(id)\,\mathrm{append}(id)\,\mathrm{failed}(id)\,\mathrm{fail}(id)),\\
L_{\mathrm{store}} &= \mathrm{pref}(\mathrm{append}(id)\,\mathrm{flush_ok}(id)\,\mathrm{committed}(id)) \cup \mathrm{pref}(\mathrm{append}(id)\,\mathrm{flush_fail}(id)\,\mathrm{failed}(id)),\\
L_0 &= \mathrm{pref}(\mathrm{req}(id)\,\mathrm{commit_ok}(id)\,\mathrm{ack}(id)) \cup \mathrm{pref}(\mathrm{req}(id)\,\mathrm{commit_fail}(id)\,\mathrm{fail}(id)).
\end{aligned}
$$

LcoordLstoreL0 = pref(req(id) append(id) committed(id) ack(id)) ∪ pref(req(id) append(id) failed(id) fail(id)), = p

The prefix closure ends at `ack` or `fail`; a second `ack`, a spontaneous output, or an incompatible continuation is not in L_{coord}Lcoord. The duplicate-ack composition trace `req, append, flush_ok, committed, ack, ack` projects onto the coordinator alphabet as `req, append, committed, ack, ack`, which is not in the prefix closure L_{coord}Lcoord; the composition trace is therefore excluded by the contract.

The store boundary observation alphabet explicitly includes `flush_ok` and `flush_fail`. The proof boundary is not the projection that hides them: hiding `flush_ok` would make the good trace `append, flush_ok, committed` and the bad trace `append, committed` indistinguishable, so the durability duty would not be checkable. The contract keeps the durability observation visible.

The decomposition obligation is $K_0(\mathrm{beh}(B_{0,\,\mathrm{coord}}), \mathrm{beh}(B_{0,\,\mathrm{store}})) \subseteq \mathrm{beh}(M_0)$K0(beh(B0,coord), beh(B0,store)) ⊆ beh(M0). It holds for the single-use protocol: any composition trace is a one-request trace; its `ack` is preceded by the coordinator's `committed`, which is preceded by the store's `flush_ok`; the root's `commit_ok/ack` path matches. The `commit_fail/fail` path matches the store's `flush_fail/failed` and the coordinator's `fail`. The abstraction maps the composition trace `req, append, flush_ok, committed, ack` to `req, commit_ok, ack` and `req, append, flush_fail, failed, fail` to `req, commit_fail, fail`; the composition's trace language is contained in L_0L0. The two-request trace `req(1), append(1), flush_ok(1), committed(1), ack(1), req(2), ...` is excluded by the interface precondition and the one-request trace language. The duplicate-ack trace `req, append, flush_ok, committed, ack, ack` is excluded by the prefix closure L_{coord}Lcoord. The wiring is stateless; the coordinator is a child cell, not an oracle inside the connector.

8.4 Child machines

Section titled "8.4 Child machines"

Coordinator M_{coord}**Mcoord**: states `idle`, `appending(id)`, `ready(id)`, `done(id)`.

- `idle --req(id)--> appending(id)` with output `append(id)`;
- `appending(id) --committed(id)--> ready(id)`;
- `ready(id) --tau--> done(id)` with output `ack(id)`;
- `appending(id) --failed(id)--> done(id)` with output `fail(id)`.

Its local active intent is "no acknowledgement before a committed result." The refinement $M_{\text{coord}} \preceq B_{0,\text{coord}}$Mcoord ≤ B0,coord is a short simulation: `ack` only from `ready`, which is reached only by `committed`; the one-request clause follows from the absence of a transition out of `done`, and the trace language is exactly L_{coord}Lcoord.

Store M_{store}Mstore: states `empty`, `writing(id)`, `flushed(id)`, `committed(id)`, `failed`.

- `empty --append(id)--> writing(id)`;
- `writing(id) --flush_ok(id)--> flushed(id)`;
- `flushed(id) --tau--> committed(id)` with output `committed(id)`;
- `writing(id) --flush_fail(id)--> failed` with output `failed(id)`.
- `committed(id)` has no outgoing transition except a silent stutter with no output.

Its local active intent is "committed implies durable." The refinement $M_{\text{store}} \preceq B_{0,\text{store}}$Mstore ≤ B0,store requires that `committed` is reachable only after `flush_ok`, and that `flush_ok` is the substrate's durability event. The boundary trace is `append, flush_ok, committed` (or `append, flush_fail, failed`); `committed` is emitted only after `flush_ok`, and the trace language is exactly L_{store}Lstore.

The context premises record the choice of a write-ahead log over an in-memory queue with periodic snapshots. The premises are the durability requirement and the crash model. The rejected alternative's reopening condition is "a durable atomic append primitive is available from the substrate." The context premise is provenance; the behavioral obligation is the refinement of the contracts.

8.5 Executable leaves

Section titled "8.5 Executable leaves"

Both `coord` and `store` are leaf cells ($\text{ch}(\text{coord}) = \text{ch}(\text{store}) = \varnothing$ch(coord) = ch(store) = ∅) whose executable realization is given directly by programs:

```
P_store:
  on append(id):
    state := writing
    write(log, id)                # volatile write
    r := fsync(log)               # substrate call
    if r = 0: emit flush_ok(id); state := flushed; emit committed(id); state := committed
    else:     emit flush_fail(id); state := failed; emit failed(id)

P_coord:
  on req(id):       state := appending; emit append(id)
  on committed(id): if state = appending: state := ready; emit ack(id); state := done
  on failed(id):    if state = appending: state := done; emit fail(id)
```

The substrate contract is: `fsync` returns 0 iff the record is durable across the modeled crash; `flush_ok(id)` is emitted only on a 0 return. The realization witness for the store relates the log state to M_{store}Mstore's state: `fsync` success corresponds to the `flush_ok` transition, and the `committed` emission corresponds to `flushed -> committed`. The invariant is "a record is in `flushed` only if the corresponding `fsync` returned 0." The good trace `append, flush_ok, committed` is admitted; the bad trace `append, committed` is not, because no `flush_ok` transition occurred. The coordinator realization is the analogous event-loop simulation; `ack` is emitted on the `ready -> done` transition, after `committed`. Under the declared run-to-completion semantics, the response cannot be delivered while the request handler is still running; the program also enters `appending` before emitting `append`, so a synchronous/reentrant response would find the handler in `appending` and would not be discarded. Under the declared semantics and the stated fairness/enabling premises, the coordinator reaches `done` and emits `ack`; this is the conditional liveness claim, not part of the safety realization.

8.6 The agent's initial mistake

Section titled "8.6 The agent's initial mistake"

The agent writes the store to emit `committed` immediately after `write` returns, before `fsync`. The refinement check $P_{\text{store}} \preceq M_{\text{store}}$Pstore ≤ Mstore fails with the finite trace $\text{append}(id)$, $\text{committed}(id)$append(id), committed(id). The trace violates the durability duty: `committed` occurs without `flush_ok`. The evidence record is fail, and the store contract remains active.

Two repairs are available.

- **Downward repair at the leaf.** Add the `fsync` before `committed`. The new store program satisfies the same contract. Only the store's leaf realization obligation $\text{Impl}(\text{store})$Impl(store) changes. The leaf machine M_{store}Mstore and its incoming contract $B_{0,\,\text{store}}$B0,store are fixed (with identity edge abstraction $\alpha_{0,\,\text{store}}$α0,store matching M_{store}Mstore to $B_{0,\,\text{store}}$B0,store), so $\text{Ref}(v_0, \text{store})$Ref(v0, store) and the parent $\text{Dec}(v_0)$Dec(v0) are unchanged. By Corollary 2 (Interface Cutoff), the parent decomposition and coordinator proofs are reused without solver re-execution.
- **Upward refinement.** Strengthening a contract cannot make an incapable substrate provide durability. The upward path is one of two authorized changes: (i) introduce a supported flush protocol and implement it, with the new contract and evidence; or (ii) revise the substrate or requirement premise with an authorized disposition and reopen the store decomposition. The example assumes the substrate supports `fsync`; the upward path is for a different substrate and must be authorized and re-verified.

The version history records both paths and the condition under which the upward path is preferred. The audit trail is not rewritten: the original failed attempt, its counterexample, and the chosen repair remain in the version history.

8.7 Inspection views via cuts

Section titled "8.7 Inspection views via cuts"

The DAG structure permits human inspection through its natural root-covering cuts:

Cut	Cut Antichain CC	What the human sees	What they do not need to read
C_0C0	$\{v_0\}${v0}	the safety intent, conditional liveness premise, abstract durability state machine, context premises	any component implementation
C_1C1	$\{\text{coord}, \text{store}\}${coord, store}	coordinator and store machines, contracts including visible `flush_ok` durability observation, context premises, evidence status	the log buffer code

At cut C_1C1, the human can verify the acknowledgement protocol: `ack` only after `committed`, `committed` only after a durable flush, and the progress duties. At the leaves, they audit the program whose contract they already understand. Corollary 1 (Pointwise and Frontier Soundness) guarantees that the cut C_1C1 view is a sound abstraction of the code.

8.8 What the example demonstrates

Section titled "8.8 What the example demonstrates"

The example exercises all three failure modes:

- the wide leap is blocked because the coordinator and store interfaces must be declared before their code;
- the rationale is preserved in the context premises, and the rejected in-memory alternative has a reopening condition;
- the human inspects cut C_1C1 rather than the WAL code.

It also exercises the downward repair and the authorized upward path, and the incremental frame cutoff theorem.

---

9. Mechanization Target

Section titled "9. Mechanization Target"

The calculus is designed to be mechanized in a small dependent-type development. The target is not a proof of the entire engineering stack; it is a kernel-checked proof of the algebraic spine, with the operational and solver obligations as explicit hypotheses.

9.1 Checked Lean 4 excerpt

Section titled “9.1 Checked Lean 4 excerpt”

```
-- Core checked declarations excerpted from JirlCalculus.lean (namespace Jirl).
-- Auxiliary definitions (Proj1, Correlated, sharedBit, Realizable, nodeRequirements)
-- follow the checked development; proofs and intermediate lemmas are omitted in this excerpt.
-- Observations and behavior
structure Machine (Obs : Type) where
  observes : Pred Obs

def Refines (M A : Machine Obs) : Prop := M.observes ⊆ₚ A.observes
def Sat (M : Machine Obs) (Φ : Pred Obs) : Prop := M.observes ⊆ₚ Φ

-- Connectors and compositional monotonicity
structure Connector (ι Obs : Type) where
  compose : (ι → Machine Obs) → Machine Obs
  precongruence : ∀ M A, (∀ i, Refines (M i) (A i)) → Refines (compose M) (compose A)

def Dec (K : Connector ι Obs) (A : ι → Machine Obs) (M : Machine Obs) : Prop :=
  Refines (K.compose A) M

theorem compositional_sibling_independence (K : Connector ι Obs)
    (children contracts : ι → Machine Obs) (parent : Machine Obs) (Φ : Pred Obs)
    (sat : Sat parent Φ) (dec : Dec K contracts parent)
    (refs : ∀ i, Refines (children i) (contracts i)) :
    Refines (K.compose children) parent ∧ Sat (K.compose children) Φ

theorem projection_erases_correlation :
    Proj1 Correlated false ∧ Proj2 Correlated true ∧ ¬ Correlated (false, true)

theorem boundary_proof_substitution_cutoff [DecidableEq ι]
    (v : Node ι Obs Context Map) (i : ι) (replacement : Machine Obs)
    (sat : Sat v.parent v.property) (dec : Dec v.connector v.contracts v.parent)
    (siblings : ∀ j, j ≠ i → Refines (v.children j) (v.contracts j))
    (newRef : Refines replacement (v.contracts i)) :
    Refines (v.connector.compose (update v.children i replacement)) v.parent ∧
    Sat (v.connector.compose (update v.children i replacement)) v.property

theorem independent_concretization_insufficient :
    (∀ i, ∃ t, sharedBit.legal t ∧ sharedBit.project t = false ∧
      Realizable (bitChild i) (sharedBit.slice i t)) ∧
    ¬ (∃ t, sharedBit.legal t ∧ sharedBit.project t = false ∧
      ∀ i, Realizable (bitChild i) (sharedBit.slice i t))

theorem upward_strengthening_preserves_dec (K : Connector ι Obs)
    (stronger contracts : ι → Machine Obs) (parent : Machine Obs)
    (strengthening : ∀ i, Refines (stronger i) (contracts i))
    (dec : Dec K contracts parent) : Dec K stronger parent

-- Section 5: Contextual projection and incremental frame cutoff
def ContextualProjection (ΔP : Prop) (K : Connector ι Obs)
    (contracts : ι → Machine Obs) (R : Pred Obs) : Prop :=
  ΔP → Sat (K.compose contracts) R

theorem contextual_projection_sound {ΔP : Prop} (K : Connector ι Obs)
    (children contracts : ι → Machine Obs) (R : Pred Obs)
    (context : ΔP) (projection : ContextualProjection ΔP K contracts R)
    (refs : ∀ i, Refines (children i) (contracts i)) :
    Sat (K.compose children) R

def passThroughConnector : Connector Unit Obs where
  compose children := children ()
  precongruence _ _ refs := refs ()

theorem pass_through_projection (ΔP : Prop) (R : Pred Obs) :
    ContextualProjection ΔP passThroughConnector (fun _ => ⟨R⟩) R

def Valid (Denote : Obl → Prop) (Req : Dev → Obl → Prop) (G : Dev) : Prop :=
  ∀ q, Req G q → Denote q

theorem validity_by_reuse {Denote : Obl → Prop} {Req : Dev → Obl → Prop}
    {G G' : Dev} (old : Valid Denote Req G)
    (covered : ∀ q, Req G' q → Req G q ∨ Denote q) : Valid Denote Req G'
```

```
theorem incremental_frame_cutoff {Denote : Obl → Prop} {Req : Dev → Obl → Prop}
    {G G' : Dev} (old : Valid Denote Req G)
    (changed : ∀ q, Req G' q → ¬ Req G q → Denote q) : Valid Denote Req G'

-- Section 5: Versioned finite snapshot specialization
namespace Versioned

inductive Obligation (Cell ι Obs : Type) where
  | sat (cell : Cell) (model : Machine Obs) (property : Pred Obs)
  | dec (cell : Cell) (connector : Connector ι Obs)
      (contracts : ι → Machine Obs) (model : Machine Obs)
  | child (cell : Cell) (slot : ι) (model contract : Machine Obs)
  | impl (cell : Cell) (program model : Machine Obs)

def Denote : Obligation Cell ι Obs → Prop
  | .sat _ model property => Sat model property
  | .dec _ connector contracts model => Dec connector contracts model
  | .child _ _ model contract => Refines model contract
  | .impl _ program model => Refines program model

abbrev Snapshot (Cell ι Obs : Type) := List (Obligation Cell ι Obs)
def Req (G : Snapshot Cell ι Obs) (q : Obligation Cell ι Obs) : Prop := q ∈ G

theorem snapshot_frame_cutoff (versions : Nat → Snapshot Cell ι Obs) (t : Nat)
    (old : Valid Denote Req (versions t))
    (changed : ∀ q, q ∈ versions (t + 1) → q ∉ versions t → Denote q) :
    Valid Denote Req (versions (t + 1))

theorem node_obligations_reused (cell : Cell) (v : Node (Fin n) Obs Unit Unit)
    (i : Fin n) (replacement : Machine Obs) :
    let next := updateChild v i replacement ()
    Obligation.sat cell next.parent next.property ∈ nodeRequirements cell v ∧
    Obligation.dec cell next.connector next.contracts next.parent ∈ nodeRequirements cell v ∧
    ∀ j, j ≠ i →
      Obligation.child cell j (next.children j) (next.contracts j) ∈ nodeRequirements cell v

theorem node_update_with_leaf_valid (cell leaf : Cell)
    (v : Node (Fin n) Obs Unit Unit) (i : Fin n)
    (replacement program : Machine Obs)
    (old : Valid Denote Req (nodeRequirements cell v))
    (newRef : Refines replacement (v.contracts i))
    (newImpl : Refines program replacement) :
    Valid Denote Req (nodeRequirements cell (updateChild v i replacement ()) ++
      [.impl leaf program replacement])

theorem changed_leaf_requires_new_evidence :
    let contract : Machine Bool := ⟨fun b => b = false⟩
    let replacement : Machine Bool := ⟨fun b => b = true⟩
    let old : Snapshot Unit Unit Bool := [.impl () contract contract]
    let next : Snapshot Unit Unit Bool := [.impl () replacement contract]
    Valid Denote Req old ∧ ¬ Valid Denote Req next

end Versioned
```

The checked formalization in `JirlCalculus.lean` treats `Obs` as an arbitrary observation type parameter and proves the algebraic kernel: connector monotonicity, compositional sibling independence, joint-trajectory realizability, top-down contextual intent projection, the general incremental frame theorem, and the concrete `Versioned` finite-snapshot specialization demonstrating proof reuse and changed-leaf rejection.

In the formalization, `Layers` is a rank-indexed support evaluator proved equal to the rank-free fixed-point `Closure` by the theorem `stratified_top_down_eq_fixed_point` under the DAG acyclicity hypotheses (`rootHeads`, `strict`, and `bounded`). Furthermore, `Versioned.Obligation` covers the four-constructor verification fragment `sat | dec | child | impl`, while `Wire` and the full Refinement-DAG well-founded induction are proved on paper in Sections 3.4—3.6.

9.2 Trusted base

Section titled “9.2 Trusted base”

The kernel checks the algebra. The following remain explicit hypotheses, with their scopes recorded:

- the operational soundness of a connector’s wiring against its transformer;
- the soundness of the evidence checker for each method;
- the adequacy of the finite model to the code for a model–checking claim;
- the substrate contract for a leaf realization;
- the input–receptiveness and progress obligations for the chosen profile.

A mechanized proof cannot make these hypotheses true. It can make their use explicit and prevent the algebra from hiding them.

9.3 Size

Section titled “9.3 Size”

The mechanized development in `JirlCalculus.lean` comprises exactly 718 lines of Lean 4 with zero `sorry` tokens and zero custom axioms (relying only on Lean’s standard foundational axioms `propext`, `Classical.choice`, and `Quot.sound`). The compact size is a design objective: a reader can audit the entire correctness spine in one sitting, and a reviewer can see exactly which premises are assumed.

---

10. Related Work

Section titled “10. Related Work”

Refinement calculi. Back and von Wright’s refinement calculus and Abrial’s Event–B develop programs by stepwise refinement, with invariants and proof obligations at each step. The refinement–DAG calculus shares the stepwise discipline but makes the interface between nodes a first–class contract and adds a persistent provenance ledger. Event–B’s refinement is per–machine; the DAG calculus adds the connector and the child–contract decomposition, which is what enables independent child verification.

Temporal specification and refinement mappings. TLA+ and the Abadi—Lamport theory of refinement mappings establish implementation relations between specifications. The DAG calculus uses the same inclusion idea, specialized to an acyclic refinement graph with explicit contracts. The connector’s monotonicity is the composition law; the DAG structure is the engineering discipline that keeps the law applicable.

Interface automata and assume–guarantee. de Alfaro and Henzinger’s interface automata formalize open components with input/output variance and compatibility. The child refinement obligation’s input receptiveness is the same discipline. Assume–guarantee reasoning is powerful but fragile when assumptions are circular; the DAG calculus avoids circularity by making the connector own the interaction and by requiring an executable coordinator rather than a circular assumption.

Design by contract. Meyer’s contracts specify preconditions, postconditions, and invariants. The DAG calculus extends the idea from a single object boundary to an acyclic decomposition: the contract is the interface between cells, and its satisfaction is checked by refinement rather than by runtime assertion alone.

Hierarchical state machines. Harel’s statecharts provide nested and orthogonal states. The DAG calculus uses ordinary state machines at each cell and a connector for interaction; it does not rely on statechart semantics. The hierarchy is in the refinement DAG structure, not in a single expanded automaton.

Counterexample–guided abstraction refinement. Clarke et al.’s CEGAR loop uses counterexamples to refine abstractions. The evolution loop is a localized CEGAR: the failure cell localizes the repair, and the shared–trajectory check separates genuine from spurious witnesses. The key difference is that cell contracts are persistent interfaces, so a repair at one cell reuses the proofs at unchanged boundaries.

Truth maintenance and provenance. Truth maintenance systems and provenance semirings model support and retraction. The provenance record borrows the idea of explicit support and reopening, but rejects general nonmonotonic dependency inference. Selection is explicit, alternatives are preserved with their reopening triggers, and support is strictly acyclic, so the warrant is evaluated in a single forward pass. The simplification is deliberate: the paper needs impact analysis and reopening, not a general truth maintenance subsystem.

Persistent data structures. Driscoll et al. established path copying and persistence. The version model represents evolution as a sequence of immutable Refinement DAGs $\{G_t\}_{t \in N}${Gt}t∈N; the incremental frame theorem establishes that unchanged obligation propositions are reused by identity, and persistent outgoing-edge containers provide an efficient store.

Assurance cases and evidential tool buses. Structured assurance cases provide explicit arguments that a critical system satisfies its top-level safety claims, standardized in frameworks such as the Goal Structuring Notation (GSN) [24] and the OMG Structured Assurance Case Metamodel (SACM) [25]. Rushby's *Evidential Tool Bus (ETB)* [26] formalizes verification tool interaction as a distributed evidence-producing architecture. Bloomfield and Rushby [27, 28] formalize assurance cases via defeaters, inductive step confidence, and epistemic probability. In the context of LLM and autonomous agent code generation, recent systems by Chechik et al.—including *CLARISSA* [29], *PLACIDUS* [30], and *AssureCode* [31]—employ assurance cases to monitor agent synthesis, construct safety envelopes, and verify agent-generated artifacts. The refinement-DAG calculus directly complements these frameworks by supplying the underlying software development with a rigorous behavioral foundation: every claim in an assurance case maps directly to a typed behavioral-inclusion proposition (SatSat, WireWire, DecDec, RefRef, ImplImpl), and the incremental frame theorem provides the exact algebraic conditions under which existing evidential branches remain valid when agent implementations change.

Agent software engineering. Recent work on repository-level coding agents (SWE-bench and related benchmarks) and on self-refinement and reflection shows that agents can iterate on code and tests. The DAG calculus addresses a different point in the loop: it constrains what an agent may accept as a completed step. It is complementary to better code generation, better test generation, and better retrieval; it supplies the missing intermediate behavioral obligations.

---

11. Limitations and Open Problems

Section titled "11. Limitations and Open Problems"

Connector admissibility. The soundness theorem is only as strong as the connector's operational soundness and reduction preservation. The theorem is conditional on validity conditions 5 and 6 (executable realization and profile admissibility); those conditions are part of DAG validity, not background assumptions. Rich concurrency — weak memory, Byzantine faults, real-time deadlines — needs richer profiles and correspondingly richer admissibility proofs. The paper's trace kernel is the base case, not the final word.

Liveness and profiles. The worked example's safety claim is discharged by the trace kernel; its liveness claim is conditional on the stated fairness and enabling premises and is not derived from the safety proof. A liveness profile needs its own operational composition law and progress-preservation witness, not a relabeling of the safety argument.

Intent correctness. The theorem guarantees that the executable system refines the admitted root intents. It does not guarantee that the root intents are the right requirements. Root authority is an organizational decision; the calculus makes it explicit and auditable, but it cannot make it correct.

Evidence soundness. The gate trusts the evidence checker. A buggy model checker, an inadequate model, or a mis-specified substrate contract can pass a wrong system. The trusted base must be stated and, where possible, minimized. The mechanization target makes the algebraic part small; the operational part remains a hypothesis.

Coverage of context premises. The gate can require explicit context premises for every interface and contract choice, but it cannot force an agent to record every implicit choice inside a leaf implementation. The provenance coverage is the declared obligations and interfaces. A leaf is a trust boundary; its contract is what the parent knows.

Test weakening. The gate blocks weakening of protected intents. It does not prevent an agent from writing a weak test for a property that was never formalized. The remedy is to formalize the property as an intent; the framework makes that remedy visible, not automatic.

Scale. The inspection view of a large DAG can still be large. The calculus bounds the change region by the dependency cone up to proved unchanged boundaries, and it bounds the size of a local proof, but a root-obligation change can reach the whole system, and it does not bound the number of cells or the cost of a single model-checking query. Tool support, proof caching, and model-checking engineering remain necessary.

Empirical validation. The claim that the protocol reduces agent error rates is a hypothesis. The paper's contribution is the calculus and the gate; a controlled study with arm-independent ground truth is the natural next step.

Decomposition depth and placement of contracts. The DAG topology and the placement of contracts are design decisions. A poor decomposition can make the connector large or the contracts weak. The calculus provides the correctness condition; the architecture remains an engineering choice, guided by the capacity and factoring remarks.

---

12. Conclusion

Section titled “12. Conclusion”

Software development by agents fails when the gap between a requirement and code is filled by unverified invention. The refinement–DAG calculus closes the gap with explicit intermediate objects: active intents with support, context premises with rationale and reopening conditions, state machines with contracts, and evidence for every behavioral obligation. One relation — behavior inclusion under typed edge abstractions — carries satisfaction, decomposition, edge refinement, and leaf realization. Stratification is the partial–order acyclicity of the Refinement DAG: it makes support evaluation an acyclic topological evaluation, makes sibling verification a substitution lemma, enables modular inspection via predecessor/successor closures and root–covering cuts, and makes version evolution reuse every proof at an unchanged interface. The soundness theorem is short because the system is one chain of inclusions, not three bolted–together worlds. The protocol is the operational reading of that chain: read the context, fix the interface, verify the machine, decompose into contracts, substitute the child proof, publish atomically. The durable–acknowledgement example shows the chain catching the exact error that motivates the framework: an agent that treats a volatile write as durable, and a gate that refuses to let it pass.

Appendix A. Proof Details

Section titled “Appendix A. Proof Details”

A.1 Theorem 1 (Refinement–DAG Soundness), expanded

Section titled “A.1 Theorem 1 (Refinement–DAG Soundness), expanded”

The executable sub–development rooted at a cell $v \in V$v ∈ V is defined by structural recursion on the refinement DAG:

- if $v \in V_{\text{leaf}}$v ∈ Vleaf, it is $\text{Beh}_{\text{exec}}(v) = \text{beh}(P_v)$Behexec(v) = beh(Pv) with its declared semantics;
- if $v \notin V_{\text{leaf}}$v ∉/ Vleaf, it is $\text{Beh}_{\text{exec}}(v)$Behexec(v), the behavior of the concrete wiring C_vCv applied to the executable sub–developments of the children $c \in \text{ch}(v)$c ∈ ch(v).

By well–founded induction on (V, ch)(V , ch), the induction hypothesis gives, for each child $c \in \text{ch}(v)$c ∈ ch(v), $B_c = \text{Beh}_{\text{exec}}(c) \subseteq \text{beh}(M_c)$ Bc = Behexec(c) ⊆ beh(Mc). Condition 5 gives $\text{Beh}_{\text{exec}}(v) \subseteq K_v(\{\alpha_{v,c}(B_c)\}_c)$Behexec(v) ⊆ Kv({αv,c(Bc)}c). The induction hypothesis and monotonicity of K_vKv give $K_v(\{\alpha_{v,c}(B_c)\}_c) \subseteq K_v(\{\alpha_{v,c}(\text{beh}(M_c))\}_c)$Kv({αv,c(Bc)}c) ⊆ Kv({αv,c(beh(Mc))}c). Edge refinement gives $\alpha_{v,c}(\text{beh}(M_c)) \subseteq \text{beh}(B_{v,c})$αv,c(beh(Mc)) ⊆ beh(Bv,c), and monotonicity gives $K_v(\{\alpha_{v,c}(\text{beh}(M_c))\}_c) \subseteq K_v(\{\text{beh}(B_{v,c})\}_c)$Kv({αv,c(beh(Mc))}c) ⊆ Kv({beh(Bv,c)}c). The decomposition obligation gives $K_v(\{\text{beh}(B_{v,c})\}_c) \subseteq \text{beh}(M_v)$Kv({beh(Bv,c)}c) ⊆ beh(Mv). Transitivity of inclusion gives $\text{Beh}_{\text{exec}}(v) \subseteq \text{beh}(M_v)$Behexec(v) ⊆ beh(Mv). The satisfaction evidence gives $\text{beh}(M_v) \subseteq [\![c]\!]$beh(Mv) ⊆ [[c]] for every active intent $c \in A(v)$c ∈ A(v). Condition 6 supplies initialization, input receptiveness, and (where claimed) progress. For any root $r \in V_{\text{root}}$r ∈ Vroot, this yields $\text{Beh}_{\text{exec}}(r) \subseteq \Phi(r) = \bigcap_{c \in A(r)} [\![c]\!]$Behexec(r) ⊆ Φ(r) = ∩c∈A(r)[[c]].

A.2 Theorem 3 (Incremental Frame and Cutoff), expanded

Section titled “A.2 Theorem 3 (Incremental Frame and Cutoff), expanded”

The obligation set $\text{Req}(G_t)$Req(Gt) is the finite set of exact, scoped behavioral–inclusion propositions generated by the DAG validity rules (§3.4–3.5). Each proposition $q \in \text{Req}(G_t)$q ∈ Req(Gt) is determined entirely by its semantic operands: $\text{Sat}(v, c)$Sat(v, c) depends on $(M_v^{(t)}, [\![c]\!], \Gamma_v^{(t)})$(Mv(t), [[c]], Γv(t)); $\text{Wire}(v)$Wire(v) depends on $(C_{t,v}, K_{t,v}, (\alpha_{t,v,c})_{c \in \text{ch}_t(v)}, X_{t,v,c}, \Gamma_{t,c}, \Gamma_v^{(t)})$(Ct,v, Kt,v, (αt,v,c)c∈cht(v), Xt,v,c , Γt,c, Γv(t)); $\text{Dec}(v)$Dec(v) depends on $(W_{t,v}, (B_{t,v,c})_{c \in \text{ch}_t(v)}, M_v^{(t)}, \Gamma_v^{(t)})$(Wt,v, (Bt,v,c)c∈cht(v), Mv(t), Γv(t)); $\text{Ref}(v, c)$Ref(v, c) depends on $(M_c^{(t)}, B_{t,v,c}, \alpha_{t,v,c}, \Gamma_{t,c})$(Mc(t), Bt,v,c, αt,v,c, Γt,c); and $\text{Impl}(\ell)$Impl(ℓ) depends on $(P_\ell^{(t)}, M_\ell^{(t)}, \Gamma_\ell^{(t)})$(Pℓ(t), Mℓ(t), Γℓ(t)).

When replacing a sub-development rooted at child $c \in \mathrm{ch}_t(v)$c ∈ cht(v) while preserving the parent machine M_vMv, wiring W_vWv, contract $B_{v,c}$Bv,c, abstraction $\alpha_{v,c}$αv,c, contexts, and sibling contracts, every proposition in the ancestor cone and sibling set belongs to $\mathrm{Req}(G_{t+1}) \cap \mathrm{Req}(G_t)$Req(Gt+1) ∩ Req(Gt). By the incremental frame theorem, because semantic operands are identical, the valid evidence record $E_t(q)$Et(q) carried over by identity into $E_{t+1}(q)$Et+1(q) remains valid and applicable, establishing $\mathrm{Holds}_{t+1}(q)$Holdst+1(q) without solver re-execution. Only newly introduced propositions in $\mathrm{Req}(G_{t+1}) \setminus \mathrm{Req}(G_t)$Req(Gt+1) ∖ Req(Gt) (such as leaf implementations $P_\ell^{\prime(t+1)}$Pℓ′(t+1) or child machines $M_c^{\prime(t+1)}$Mc′(t+1)) require evaluation. When these hold ($\mathrm{Holds}_{t+1}(q)$Holdst+1(q)), G_{t+1} Gt+1 is unconditionally valid, and Theorem 1 guarantees root behavioral refinement without ancestor solver re-execution.

A.3 Lemma 3, expanded

Section titled “A.3 Lemma 3, expanded”

Let $\rho = s_0 \xrightarrow{\sigma_1} s_1 \xrightarrow{\sigma_2} \cdots \xrightarrow{\sigma_n} s_n$ρ = s0 σ1

---

s1 σ2

⋯ σn

---

sn be a child execution. Initiation relates s_0s0 to an initial contract state t_0t0. Inductively, $s_i$si is in I_cIc and related to $t_i$ti. For step $i+1$ i + 1: if $\lambda_{v,c}(\sigma_{i+1}) = \epsilon$λv,c(σi+1) = ϵ, the relation keeps $t_{i+1} = t_i$ti+1 = ti; otherwise the step-simulation condition supplies t_{i+1}ti+1 with the matching contract action. At the end, the sequence of non-silent contract actions is exactly $\lambda_{v,c}(\rho)$λv,c(ρ), so the abstract trace is in $\mathrm{tr}(B_{v,c})$tr(Bv,c). Input receptiveness ensures the abstract trace is not vacuous at open inputs; silent convergence ensures liveness profiles do not hide infinite internal stuttering.

Appendix B. Notation

Section titled “Appendix B. Notation”

Symbol	Meaning
VV	finite set of cells in the Refinement DAG
V_{root}Vroot	root cells of the DAG ($\{r \in V \mid \mathrm{par}(r) = \varnothing\}${r ∈ V ∣ par(r) = ∅})
V_{leaf}Vleaf	leaf cells of the DAG ($\{\ell \in V \mid \mathrm{ch}(\ell) = \varnothing\}${ℓ ∈ V ∣ ch(ℓ) = ∅})
$\mathrm{ch}(v), \mathrm{par}(v)$ch(v), par(v)	direct child cells and direct parent cells of vv
$\mathrm{Anc}(S), \mathrm{Desc}(S)$Anc(S), Desc(S)	predecessor (ancestor) closure and successor (descendant) closure on (V, ch) (V , ch)
CC	root-covering cut (frontier antichain intersecting every path from V_{root}Vroot to V_{leaf} Vleaf)
O_XOX	observation space at interface XX
$\mathrm{beh}(M)$beh(M)	behavior set of machine MM
$\mathrm{Beh}_{\mathrm{exec}}(v)$Behexec(v)	executable behavior of the sub-development rooted at vv
C_vCv	concrete wiring implementation at cell vv
Q_cQc	an admissible child behavior used in the wiring-adequacy quantifier
$\preceq$ ⪯, $\preceq_\alpha$ ⪯α	refinement (with abstraction αα)
I, JI, J	intent records and acyclic support links
$M(v)$M(v)	local model of cell vv ($O_{X_v}, \Phi(v), M_v, \Gamma_v$OXv , Φ(v), Mv, Γv)

Symbol	Meaning
$\Delta P_v, \Delta P_v^*$ΔPv, ΔPv∗	local context premises of cell vv and their transitive upstream closure
M_vMv	machine of cell vv
$A, A(v)$A, A(v)	active intents in development, owned by cell vv
$\Phi(v)$Φ(v)	semantic conjunction of active intents owned by vv
$\chi(v)$χ(v)	decomposition record $(W_v, \mathrm{ch}(v), (B_{v,c}, \alpha_{v,c})_c)$(Wv, ch(v), (Bv,c, αv,c)c) or leaf program P_vPv
W_vWv	wiring skeleton at cell vv
K_vKv	connector transformer at cell vv
$B_{v,c}$Bv,c	slot contract from vv to child cc at slot interface $X_{v,c}$Xv,c
$\alpha_{v,c}$αv,c	typed edge abstraction map from child observations O_{X_c}OXc to slot interface $O_{X_{v,c}}$ OXv,c
EE	evidence map
R_vRv	component requirement at cell vv
$\mathrm{Req}(G_t)$Req(Gt)	finite set of exact behavioral–inclusion propositions for G_tGt ($\mathrm{Sat}, \mathrm{Wire}, \mathrm{Dec}, \mathrm{Ref}, \mathrm{ImplSat}, \mathrm{Wire}, \mathrm{Dec}, \mathrm{Ref}, \mathrm{Impl}$)
G_tGt	versioned Refinement DAG at epoch tt
$\mathrm{stale}(x)$stale(x)	downstream cells depending on upstream item xx along ΔP_v^*ΔPv∗
$\mathrm{realizable}(M, o)$realizable(M, o)	observation oo is realizable by machine MM ($o \in \mathrm{beh}(M)$o ∈ beh(M))
$\mathrm{pref}(L)$pref(L)	prefix closure of trace set LL ($\{u \mid \exists w.\ uw \in L\}${u ∣ ∃w. uw ∈ L})
$\mathrm{Holds}_t(q)$Holdst(q)	proposition $q \in \mathrm{Req}(G_t)$q ∈ Req(Gt) is mathematically satisfied and witnessed by valid evidence in E_tEt
$X_{t,v}, X_{t,v,c}$Xt,v, Xt,v,c	cell interface and slot interface at version tt
$\Gamma_{t,v}, \Gamma_{t,c}$Γt,v, Γt,c	semantic profile and context for cell vv and child cc at version tt
$C_{t,v}, K_{t,v}$Ct,v, Kt,v	concrete wiring implementation and transformer at cell vv at version tt

---

References

Section titled “References”

1. J.–R. Abrial. *The B–Book: Assigning Programs to Meanings.* Cambridge University Press, 1996.
2. R.–J. Back and J. von Wright. *Refinement Calculus: A Systematic Introduction.* Springer, 1998.
3. M. Abadi and L. Lamport. Conjoining Specifications. *ACM Transactions on Programming Languages and Systems*, 17(3):507—534, 1995.
4. M. Abadi and L. Lamport. The Existence of Refinement Mappings. *Theoretical Computer Science*, 82(2):253—284, 1991.
5. L. de Alfaro and T. A. Henzinger. Interface Automata. In *Proc. ESEC/FSE*, pages 109—120, 2001.
6. E. M. Clarke, O. Grumberg, S. Jha, Y. Lu, and H. Veith. Counterexample–Guided Abstraction Refinement for Symbolic Model Checking. *Journal of the ACM*, 50(5):752—794, 2003.
7. J. de Kleer. An Assumption–Based Truth Maintenance System. *Artificial Intelligence*, 28(2):127—162, 1986.
8. J. R. Driscoll, N. Sarnak, D. D. Sleator, and R. E. Tarjan. Making Data Structures Persistent. *Journal of Computer and System Sciences*, 38(1):86—124, 1989.
9. T. J. Green, G. Karvounarakis, and V. Tannen. Provenance Semirings. In *Proc. PODS*, pages 31—40, 2007.
10. D. Harel. Statecharts: A Visual Formalism for Complex Systems. *Science of Computer Programming*, 8(3):231—274, 1987.
11. M. P. Herlihy and J. M. Wing. Linearizability: A Correctness Condition for Concurrent Objects. *ACM Transactions on Programming Languages and Systems*, 12(3):463—492, 1990.

12. C. A. R. Hoare. *Communicating Sequential Processes.* Prentice Hall, 1985.
13. C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? In *Proc. ICLR*, 2024.
14. W. Kunz and H. W. J. Rittel. Issues as Elements of Information Systems. Working Paper 131, University of California, Berkeley, 1970.
15. L. Lamport. *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers.* Addison-Wesley, 2002.
16. R. J. Lipton. Reduction: A Method of Proving Properties of Parallel Programs. *Communications of the ACM*, 18(12):717—721, 1975.
17. A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegreffe, U. Alon, N. Dziri, S. Prabhumoye, Y. Yang, S. Gupta, B. P. Majumder, K. Hermann, S. Welleck, A. Yazdanbakhsh, and P. Clark. Self-Refine: Iterative Refinement with Self-Feedback. In *Proc. NeurIPS*, 2023.
18. D. McAllester. Truth Maintenance. In *Proc. AAAI*, pages 1109—1116, 1990.
19. B. Meyer. *Object-Oriented Software Construction.* Prentice Hall, 2nd edition, 1997.
20. N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao. Reflexion: Language Agents with Verbal Reinforcement Learning. In *Proc. NeurIPS*, 2023.
21. N. P. Suh. *Axiomatic Design: Advances and Applications.* Oxford University Press, 2001.
22. A. van Lamsweerde. Goal-Oriented Requirements Engineering: A Guided Tour. In *Proc. IEEE RE*, pages 249—262, 2001.
23. S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao. ReAct: Synergizing Reasoning and Acting in Language Models. In *Proc. ICLR*, 2023.
24. Goal Structuring Notation Working Group. *GSN Community Standard Version 3.* SCSC, 2021.
25. Object Management Group. *Structured Assurance Case Metamodel (SACM) Version 2.3.* OMG, 2021.
26. J. Rushby. The Evidential Tool Bus: A Software Architecture for Automated Verification. In *Proc. FMICS*, pages 1—11, 2005.
27. R. Bloomfield and J. Rushby. Assurance Cases, Defeaters, and Epistemic Confidence. *IEEE Transactions on Software Engineering*, 50(6):1452—1468, 2024.
28. R. Bloomfield and J. Rushby. Defeaters in Assurance Cases. In *Proc. DSN*, pages 310—322, 2025.
29. M. Chechik et al. CLARISSA: Continuous Layered Assurance for Robotic and Autonomous Systems. In *Proc. ICSE-SEIP*, 2024.
30. M. Chechik et al. PLACIDUS: Pluggable Assurance for LLM-Generated Cyber-Physical Systems. In *Proc. ASE*, 2025.
31. M. Chechik et al. AssureCode: Dynamic Assurance Cases for Autonomous Coding Agents. In *Proc. ICSE*, 2026.